MBC AI 데이터 저널리즘 · 김형석

Olist 이커머스 데이터 분석과 AI 서비스 실습 포트폴리오

데이터를 연결하고, 모델을 비교하고, 근거를 확인한 과정

실습 자료 기준 2026.11.05 · 문서화 및 추가 확인 2026.11.06

96,203건
분석 주문

R$15.37M
결제 금액 합계

8.1%
배송 지연율

3.0%
재구매 고객 비율



재생성한 확인용 분석 파일을 사용한 실제 Streamlit 화면. 결제 합계의 정확한 값은 R$15,374,496.52다.

고객·주문·상품·결제·리뷰 데이터를 연결하고, 추천·감성·판매자 모델을 비교한 뒤 웹 화면과 RAG 근거 검증으로 확장한 실습이다.

강사 제공 코드와 과거 저장 출력, 이번 문서화의 재계산·화면 확인을 구분했다. 타인의 참고 결과는 본인 성과로 사용하지 않았다.

문서 구성

전체 실습 흐름과 읽는 순서

수업은 데이터 분석 결과를 실제 사용할 수 있는 화면과 질의응답으로 확장하는 흐름으로 구성돼 있다. 단계별로 목적, 입력, 실제 코드, 실행 순서, 결과, 해석을 연결했다.

각 STEP은 목적→필요한 이유→기술과 코드→실습 과정→실행 결과→해석→배운 점의 순서로 읽는다. 표와 그래프가 있는 단계는 설명 바로 뒤에 확대 증빙을 배치했다.

단계	실습 내용	확인할 산출물
STEP1 01~05	CSV 준비·이해·병합·EDA	통합 CSV, 분석 지표와 그래프
STEP1 06~07	상품 추천·리뷰 감성 ML DL BERT	추천 모델과 감성 모델, 저장 평가
STEP1 08~10	판매자 유형·이탈 예측·전략	군집, 이탈 모델, seller_table
STEP2	Gradio로 저장 모델 비교	입력 문장과 모델별 긍정 확률
STEP3	로컬 Streamlit 대시보드	7개 메뉴와 분석 Parquet
STEP4	자료 검색·답변·근거 대조	사실표, 설명서, 규정집, 답변 ID

증거의 상태를 구분하는 방법

표시	의미
코드 확인	해당 구현이 실제 파일에 존재함
저장 출력	노트북에 남아 있는 과거 실행 결과. 실행 주체와 모든 조건은 별도 입증하지 않음
재계산	문서화 과정에서 CSV 또는 저장 모델로 다시 계산함
화면 확인	문서화 중 실제 웹 화면에서 확인하고 캡처함
기존 제출 화면	평가 Word에 삽입된 화면. 이번 API 실행과 구분
미확인	새 실행 또는 객관적 증거를 아직 확보하지 못함

타인의 참고 이미지 17장은 구성과 시각적 표현을 참고하는 자료다. 타인의 수치, 서비스 URL, 실행 화면을 본인 성과로 사용하지 않았다.

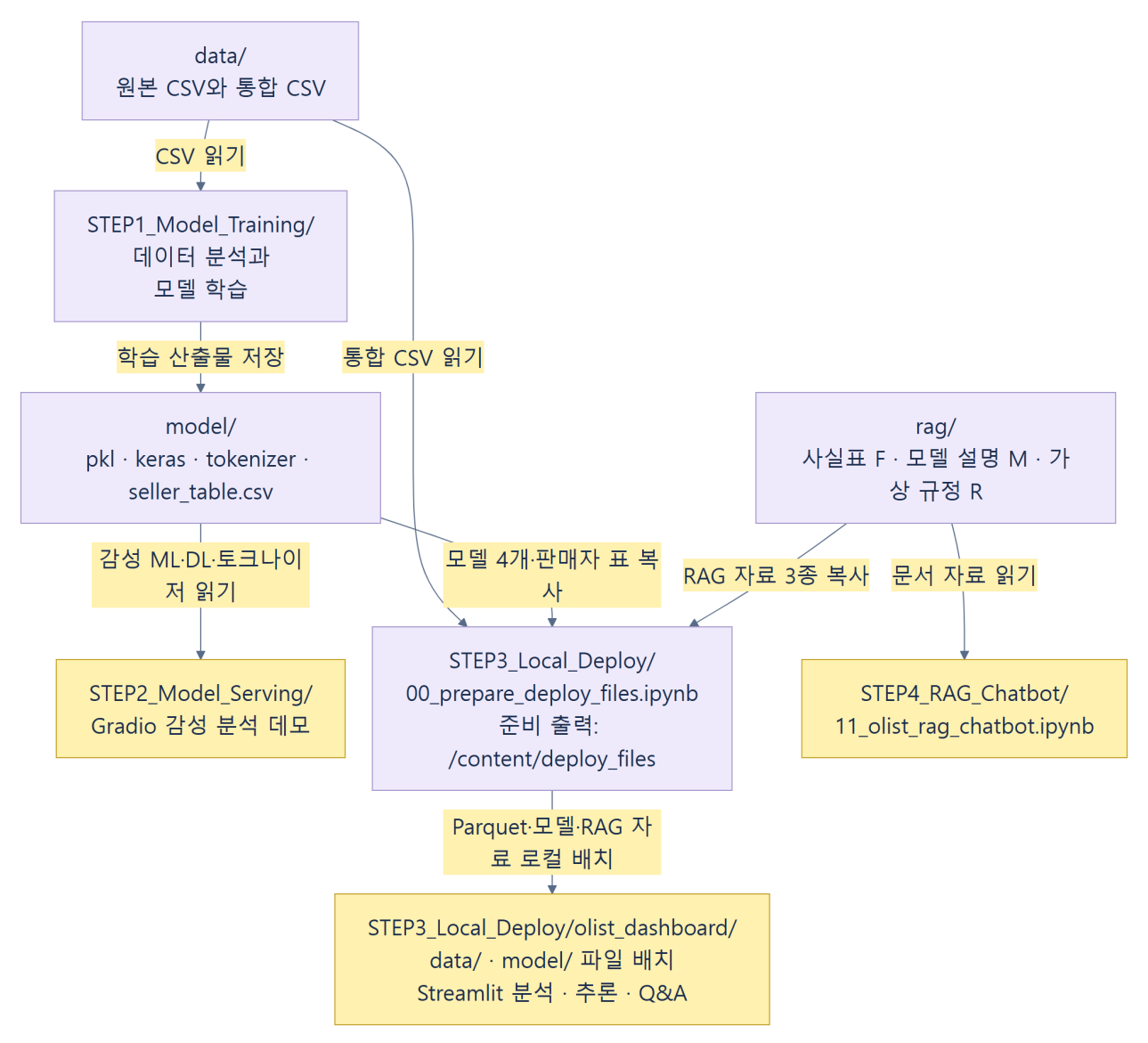
코드를 읽는 방법

코드 상자는 원본의 일부를 설명하기 위한 발췌다. import, 변수 준비, 함수의 앞뒤 또는 괄호가 생략될 수 있어 단독 실행용 전체 코드가 아니다. 표시된 파일과 셀에서 앞뒤 코드를 함께 읽어야 한다. 발췌 끝은 해당 함수나 실습의 종료를 뜻하지 않는다.

구조도 01 전체 프로젝트

데이터와 저장 모델이 이동하는 경로

실선 화살표는 CSV 읽기, 모델 저장·읽기, 배포 파일 복사·배치를 뜻한다. Gradio와 Streamlit은 서로 독립된 실행 경로다.



같은 Mermaid 원본에서 로컬 렌더링한 고해상도 구조도. 화살표와 관계는 실제 파일·코드를 대조했다.

rag/ 자료는 기존에 제공된 문서다. 모델 예측 결과가 자동으로 RAG 문서가 되는 연결은 그리지 않았다. 화살표는 코드의 입출력 경로이며 이번 작업에서 모든 경로를 새로 실행했다는 뜻은 아니다.

근거: STEP1 저장 셀, STEP2 감성 데모, STEP3 준비 노트북 셀 4·5, STEP4 셀 3·5·7·9, app.py와 utils의 읽기 경로. 준비 출력은 Colab /content/deploy_files이며 로컬 dashboard 배치와 구분한다.

STEP 01 자료 확보

원본 자료와 실행 환경

① 실습 목적

학습한 노트북, 데이터, 모델, 웹앱이 어디에 있는지 한눈에 정리한다.

② 왜 이 작업이 필요한가

음식점에서 주문서와 영수증, 조리법을 따로 보관하듯 프로젝트도 파일의 역할을 알아야 잘못된 자료를 사용하지 않는다.

③ 사용한 기술과 코드

노트북(Notebook)은 설명·코드·실행 결과를 함께 담은 .ipynb 파일이다. 가상환경(Virtual Environment)은 이 프로젝트에 필요한 프로그램 묶음을 따로 보관하는 공간이다.

④ 실습 과정

1. Olist.zip의 핵심 폴더를 확인했다.
2. 로컬 C:/WORKSPACE/Olist와 핵심 파일 84개를 비교했다.
3. Drive 스크린샷 15장의 파일명과 구조를 대조했다. 새 분석을 실행한 단계는 아니다.

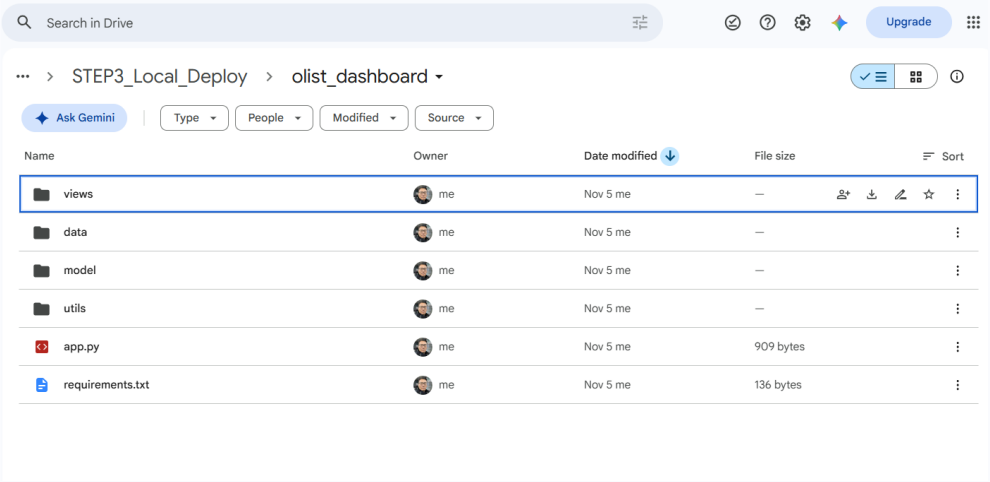
⑤ 실행 결과

자료	확인 결과
Olist.zip	33,675개 항목, 비압축 합계 1,198,272,910바이트. 가상환경과 Git 내부 파일이 다수 포함됨
ZIP 대 로컬	가상환경·Git·캐시를 제외한 핵심 파일 84개 CRC 대조 결과 일치. 최신 수정 이력이나 실행 성공까지 증명하는 것은 아님
Drive 캡처 15장	폴더와 파일명 확인. 105233과 105241은 같은 폴더 화면
참고 이미지 17장	타인 README·발표 자료의 구성 참고. 분석 결과는 전용하지 않음
평가 Word	요구사항과 삽입된 Olist RAG 화면을 확인

Python 3.12.10 / streamlit 1.66.1 / pandas 3.1.1 / numpy 2.4.6 / scikit-learn 1.6.1 / pyarrow 26.0.1 / langchain-core 1.7.6 / langchain-openai 1.7.4

Colab 노트북의 배포 준비 출력에는 Python 3.13.18이 기록돼 있지만 현재 로컬 확인 환경은 3.12.10이었다. 저장된 과거 환경과 현재 환경을 같은 것으로 쓰지 않았다.

Drive 파일 목록



Google Drive의 olist_dashboard 폴더. app.py와 views·utils·data·model의 분리를 확인했다.

⑥ 결과 해석

핵심 파일이 일치한다는 것은 같은 자료를 살펴봤다는 뜻이다. 그 파일이 정상 실행되거나 내용이 맞다는 뜻까지는 아니다.

⑦ 이번 단계에서 배운 점

실습 결과를 설명하려면 먼저 어느 파일을 근거로 삼았는지 알아야 한다. 과거 Colab 환경과 현재 로컬 확인 환경도 구분해 기록해야 한다.

STEP 02 데이터 이해

데이터를 불러오고 구조를 확인한 과정

① 실습 목적

Olist의 각 표가 무엇을 담고 있는지 읽고, 빈칸과 연결 번호를 확인한다.

② 왜 이 작업이 필요한가

주문서·메뉴 목록·결제 영수증은 서로 다른 표다. 각각을 구분해야 주문 수와 결제 금액을 올바르게 셀 수 있다.

③ 사용한 기술과 코드

CSV(Comma Separated Values)는 쉼표로 칸을 구분해 저장한 표 파일이다. pandas는 Python에서 표를 읽고 정리하는 도구이며, 읽어 온 표를 데이터프레임(DataFrame)이라고 한다. pd.read_csv는 CSV를 표로 가져온다.

```
# 해설: CSV를 Python의 표로 읽는다.
# 셀 3. 9개 CSV 파일 모두 불러오기
# PART 2에서 저장한 파일들을 불러옵니다.

customers   = pd.read_csv(f"{base_path}/olist_customers_dataset.csv")
orders      = pd.read_csv(f"{base_path}/olist_orders_dataset.csv")
order_items = pd.read_csv(f"{base_path}/olist_order_items_dataset.csv")
products    = pd.read_csv(f"{base_path}/olist_products_dataset.csv")
payments    = pd.read_csv(f"{base_path}/olist_order_payments_dataset.csv")
reviews     = pd.read_csv(f"{base_path}/olist_order_reviews_dataset.csv")
sellers     = pd.read_csv(f"{base_path}/olist_sellers_dataset.csv")
geolocation = pd.read_csv(f"{base_path}/olist_geolocation_dataset.csv")
category    = pd.read_csv(f"{base_path}/product_category_name_translation.csv")
```

핵심 코드 발췌 · 앞뒤 코드 생략
STEP1_Model_Training/03_data_understanding.ipynb · 셀 4 · 일부 발췌 · 해설 주석 추가, 계산 로직 유지

④ 실습 과정

1. 02_data_loading.ipynb에서 Drive 연결과 CSV 9종 준비 기록을 확인했다.
2. 03_data_understanding.ipynb 셀 4에서 표를 읽는 코드를 확인했다.
3. 셀 5의 크기와 셀 9의 결측치 출력을 로컬 CSV 검사와 대조했다.

⑤ 실행 결과

표	행	열	분석에서의 역할
orders	99,441	8	주문 한 건과 상태·날짜
customers	99,441	5	주문 고객과 실제 고객의 구분
order_items	112,650	7	한 주문에 담긴 각 상품
payments	103,886	5	분할 결제를 포함한 결제 내역
reviews	99,224	7	별점과 리뷰 문장
products	32,951	9	상품 특징
sellers	3,095	4	판매자 정보
geolocation	1,000,163	5	위치 정보
category	71	2	카테고리 번역

저장 출력과 로컬 CSV 검사를 대조했다. reviews의 review_comment_message는 58,247개, review_comment_title은 87,656개가 비어 있다. 글이 없는 리뷰를 감성 학습에서 그대로 사용할 수 없으므로 후속 전처리가 필요하다.

⑥ 결과 해석

리뷰 글이 비어 있는 행은 감성 분석의 입력으로 사용할 수 없다. 반면 별점만 있는 리뷰는 별점 분포 분석에는 활용할 수 있다. 분석 목적에 따라 필요한 열이 다르다.

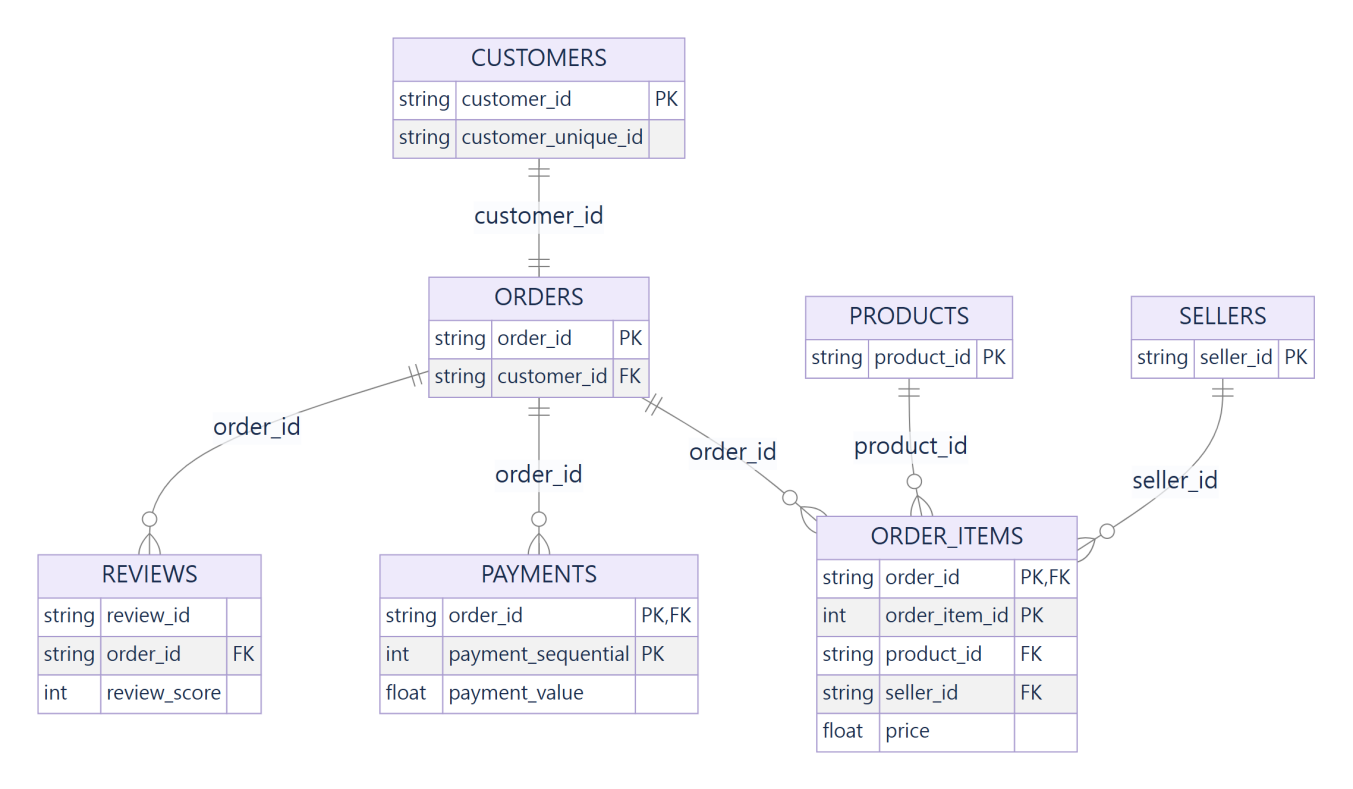
⑦ 이번 단계에서 배운 점

표의 행은 기록 한 줄, 열은 정보의 종류다. customer_id와 customer_unique_id는 역할이 달라 재구매 고객을 셀 때는 실제 사람을 구분하는 후자를 사용한다.

구조도 02 데이터 병합

연결 키와 집계 단위를 구분하는 관계도

CSV 7종을 실제로 읽어 관계를 확인했다. PK는 식별 키, FK는 연결 키를 뜻하는 논리적 표시이며 데이터베이스에 제약조건을 생성한 것은 아니다. 1은 한 행, 0..N은 없거나 여러 행을 뜻한다.



같은 Mermaid 원본에서 로컬 렌더링한 고해상도 구조도. 화살표와 관계는 실제 파일·코드를 대조했다.

현재 customers와 orders는 customer_id 기준 99,441개가 각각 한 번씩 있어 1:1이다. 같은 실제 사람의 여러 주문은 customer_unique_id로 묶는다. reviews의 review_id는 고유하지 않아 PK로 표시하지 않았다.

집계 대상	한 행의 단위와 안전한 집계
주문 수	order_id 고유 개수. 병합된 전체 행 수를 주문 수로 세지 않음
주문 상품	order_id + order_item_id. price는 상품 행 기준
결제 금액	order_id + payment_sequential 중복 제거 후 주문별 payment_value 합산
리뷰·재구매	리뷰 비교는 기존 주문별 첫 리뷰 기준 유지. 재구매는 customer_unique_id별 고유 주문 수

예를 들어 상품 3행과 결제 2행을 order_id로 바로 붙이면 6행이 된다. 리뷰까지 여러 행이면 더 늘어난다. 실제 원본에는 상품 여러 행 주문 9,803개, 결제 여러 행 주문 2,961개, 리뷰 여러 행 주문 547개가 있어 집계 단위를 따로 맞춰야 한다. 이 수치는 새 관계 점검이며 기존 분석 KPI의 대상·기간과 다르다.

STEP 03 데이터 병합

여러 표를 붙이고 집계 단위를 구분한 과정

① 실습 목적

흩어진 주문·고객·상품·결제·리뷰 정보를 같은 번호로 연결한다.

② 왜 이 작업이 필요한가

주문서만 보면 구매 상품의 종류나 고객 지역을 알 수 없다. 공통 번호로 다른 표를 붙여야 배송과 리뷰 같은 정보를 함께 분석할 수 있다.

③ 사용한 기술과 코드

데이터 병합(Merge)은 공통 열의 같은 값을 찾아 두 표를 붙이는 작업이다. on은 맞춰 볼 번호, how="left"는 왼쪽 표의 기록을 남기는 방식이다. drop_duplicates는 같은 기준의 중복 기록을 한 번만 남긴다.

```
# 해설: 같은 번호로 붙이고, 집계할 때 중복 단위를 맞춘다.
df = pd.merge(orders, customers, on="customer_id", how="left")
df = pd.merge(df, order_items, on="order_id", how="left")
df = pd.merge(df, products, on="product_id", how="left")
df = pd.merge(df, category, on="product_category_name", how="left")
df = pd.merge(df, payments, on="order_id", how="left")
df = pd.merge(df, reviews, on="order_id", how="left")
df = pd.merge(df, sellers, on="seller_id", how="left")
```

핵심 코드 발췌 · 앞뒤 코드 생략
04_data_merge.ipynb · 셀 10 12 14 16 18 20 22에서 병합문 발췌 · 해설 주석 추가, 계산 로직 유지

```
# 해설: 같은 번호로 붙이고, 집계할 때 중복 단위를 맞춘다.
# 셀 5-1. 분석 단위 맞추기 (중복 행 정리)
# drop_duplicates(기준) : 기준 컬럼 값이 같은 행은 첫 번째 1개만 남깁니다.
# dropna(subset=[...]) : 상품/결제 정보가 없는 주문(취소 등)은 해당 단위에서 제외합니다.

orders = df.drop_duplicates("order_id")                          # 주문 1건 = 1행
items  = (df.dropna(subset=["order_item_id"])
            .drop_duplicates(["order_id", "order_item_id"]))     # 상품 1개 = 1행
pay    = (df.dropna(subset=["payment_sequential"])
            .drop_duplicates(["order_id", "payment_sequential"])) # 결제 1건 = 1행
```

핵심 코드 발췌 · 앞뒤 코드 생략
STEP1_Model_Training/05_eda.ipynb · 셀 8 · 일부 발췌 · 해설 주석 추가, 계산 로직 유지

④ 실습 과정

1. 04_data_merge.ipynb 셀 10→12→14→16→18→20→22의 병합문을 확인했다.
2. 각 단계의 저장된 행 수 변화를 읽었다.
3. 05_eda.ipynb 셀 8에서 주문·상품·결제의 중복 제거 기준을 확인하고 CSV 규모를 다시 확인했다.

⑤ 실행 결과

단계	행 수	의미
고객 연결 후	99,441	주문 단위 유지
주문 상품 연결 후	113,425	한 주문의 여러 상품으로 증가
결제 연결 후	118,434	분할 결제와 상품의 조합
최종 통합	119,143 × 40열	리뷰와 판매자까지 연결

결과: 로컬 CSV에서 통합 119,143행×40열, 고유 주문 99,441건을 확인했다.

주문 한 건이 여러 행으로 늘어나도 주문 수는 고유 주문 번호로 센다.

⑥ 결과 해석

주문 한 건에 메뉴 3개와 결제 2개가 있으면 표를 붙인 뒤 여러 행으로 늘어날 수 있다. 통합 표의 행 수가 곧 주문 수는 아니다. geolocation은 이 병합 체인에 포함되지 않는다.

⑦ 이번 단계에서 배운 점

분석 전에 무엇을 한 건으로 셀지 정해야 한다. 주문·상품·결제의 단위를 구분하면 중복 매출과 주문 수 과장을 막을 수 있다.

STEP 04 EDA 준비

분석 기간과 지표 계산 기준을 정한 과정

① 실습 목적

어떤 주문을 분석할지 먼저 정하고, 같은 조건으로 지표를 계산한다.

② 왜 이 작업이 필요한가

취소 주문과 완료 주문을 섞거나 수집이 덜 된 기간을 비교하면 잘못된 결론을 낼 수 있다. 비교할 범위를 먼저 정해야 숫자의 뜻이 분명해진다.

③ 사용한 기술과 코드

탐색적 데이터 분석(Exploratory Data Analysis, EDA)은 표와 그래프로 데이터의 특징을 살펴보는 과정이다. 데이터 전처리(Preprocessing)는 그 전에 중복·빈칸·날짜를 정리하는 일이다. 조건식은 필요한 행만 골라내는 필터 역할을 한다.

```
# 해설: 분석 대상과 기간을 먼저 정한다.
# 셀 9-1. 분석용 기준 데이터(base) 만들기

start_date = "2017-01-05"
end_date   = "2018-08-30"   # "< 2018-08-30" → 2018-08-29 23:59:59까지 포함

base = orders[
    (orders["order_status"] == "delivered")                # 배송 완료 주문만
    & (orders["order_delivered_customer_date"].notna())    # 도착일이 있는 주문만
    & (orders["order_purchase_timestamp"] >= start_date)
    & (orders["order_purchase_timestamp"] <  end_date)
].copy()

# 주문별 결제 총액 : 결제를 나눠 낸 주문도 있으므로 pay에서 주문별로 합칩니다.
# map() : order_id를 기준으로 합계 값을 찾아 연결
base["order_value"] = base["order_id"].map(
```

핵심 코드 발췌 · 앞뒤 코드 생략
STEP1_Model_Training/05_eda.ipynb · 셀 16 · 일부 발췌 · 해설 주석 추가, 계산 로직 유지

```
# 해설: 분석 대상과 기간을 먼저 정한다.
pay = (df.dropna(subset=["payment_sequential"])
         .drop_duplicates(["order_id", "payment_sequential"]))
base["order_value"] = base["order_id"].map(
    pay.groupby("order_id")["payment_value"].sum()
)
```

핵심 코드 발췌 · 앞뒤 코드 생략
05_eda.ipynb · 셀 8과 셀 16의 집계 규칙 발췌 · 해설 주석 추가, 계산 로직 유지

④ 실습 과정

1. 05_eda.ipynb 셀 8에서 집계 단위를 맞춘다.
2. 셀 16의 배송 완료·수령일·기간 조건을 확인한다.
3. 주문별 결제 합계와 배송 일수·지연 여부를 계산하는 규칙으로 이번 문서화에서 재계산했다.

⑤ 실행 결과

정의	기준
대상	배송 완료·수령일 존재
기간	2017-01-05 이상, 2018-08-30 미만
결제 합계	주문별 분할 결제 금액을 합산
지연	실제 도착 시각 > 예정 도착 시각
리뷰	통합 CSV에서 주문별 첫 리뷰
재구매	고객별 주문 2건 이상인 고객 비율

⑥ 결과 해석

2018-08-30 미만이면 8월29일 밤까지 포함된다. 8월29일 자정 이하로 쓰면 당일 대부분이 빠진다. 이번 확인에서는 이 차이로 11건이 달라졌다.

⑦ 이번 단계에서 배운 점

분석 기간과 대상은 결과에 붙는 설명이 아니라 계산의 일부다. 날짜가 시각까지 포함한다는 점을 이해해야 같은 결과를 재현할 수 있다.

STEP 05 EDA 결과

분석 결과를 다시 계산해 확인한 내용

① 실습 목적

README와 사실표에 적힌 핵심 숫자가 실제 CSV와 같은지 확인한다.

② 왜 이 작업이 필요한가

다른 사람의 화면에 있는 숫자를 옮기면 필터나 데이터가 다를 수 있다. 제출할 숫자는 본인 자료의 기준으로 확인해야 한다.

③ 사용한 기술과 코드

핵심 성과 지표(Key Performance Indicator, KPI)는 중요한 상태를 빠르게 보는 숫자다. 이 실습에서는 주문 수, 결제 금액, 지연율과 리뷰가 해당한다. 평균은 값의 합을 개수로 나눈 것이고 비율은 전체 중 해당하는 몫이다.

```
# 해설: 같은 기준에서 지연 여부별 평균 리뷰를 비교한다.
# 셀 9. (검산) 사실표 숫자를 원본 데이터로 다시 계산해 보기
df = pd.read_csv(f"{base_path}/data/olist_master_data.csv", parse_dates=[
    "order_purchase_timestamp", "order_delivered_customer_date",
"order_estimated_delivery_date"])
o = df.drop_duplicates("order_id")
o = o[(o["order_status"] == "delivered") & o["order_delivered_customer_date"].notna()
      & (o["order_purchase_timestamp"] >= "2017-01-05") & (o["order_purchase_timestamp"]
 < "2018-08-30")]
late = o["order_delivered_customer_date"] > o["order_estimated_delivery_date"]
repeat = (o.groupby("customer_unique_id")["order_id"].count() >= 2).mean()

print(f"F01 주문 수   : {len(o):,}건")
print(f"F12 지연율    : {late.mean():.1%}")
print(f"F07 재구매    : {repeat:.1%}")
```

핵심 코드 발췌 · 앞뒤 코드 생략
STEP4_RAG_Chatbot/11_olist_rag_chatbot.ipynb · 셀 19 · 일부 발췌 · 해설 주석 추가, 계산 로직 유지

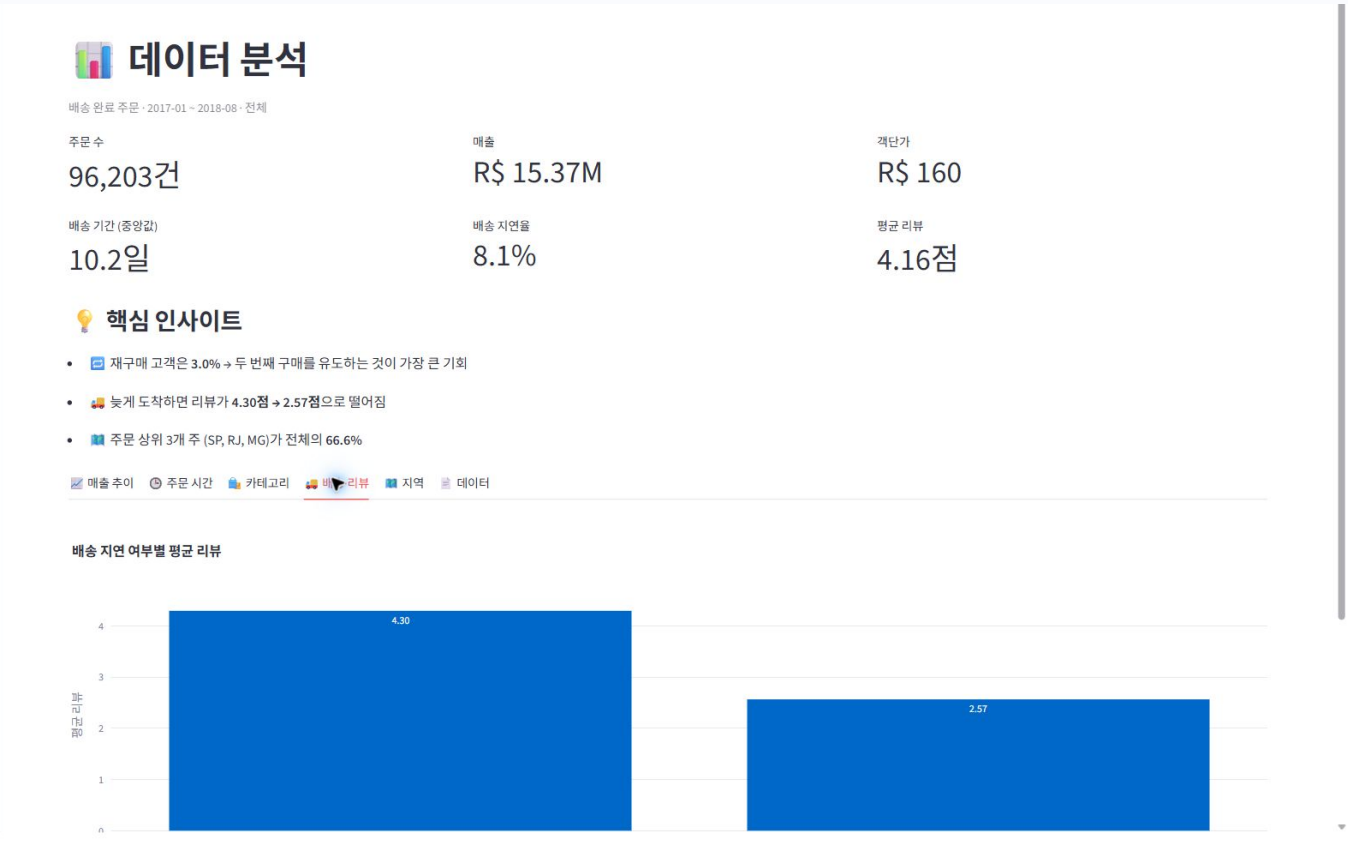
④ 실습 과정

1. 05_eda.ipynb 셀 16과 같은 분석 조건을 적용했다.
2. customer_unique_id별 주문 수로 재구매 비율을 계산했다.
3. 제때 도착한 주문과 지연 주문의 평균 리뷰를 나눠 계산하고 verified_metrics.json에 기록했다.

⑤ 실행 결과

지표	재계산 결과	표시 값
분석 주문	96,203	96,203건
결제 금액 합계	15,374,496.52 BRL	R$ 15.37M
배송 지연율	8.130724%	8.1%
재구매 고객 비율	2.995832%	3.0%
제때 도착 평균 리뷰	4.295295	4.30점
지연 도착 평균 리뷰	2.565683	2.57점

문서화 중 실제 화면 확인



이번 문서화의 실제 화면 확인. 재생성한 분석 파일의 지표이며 원본 CSV의 핵심 평균값과 대조했다.

⑥ 결과 해석

지연 주문의 평균 리뷰가 약 1.73점 낮았다. 이는 두 집단의 차이가 관찰됐다는 뜻이며, 다른 조건을 통제해 배송 지연의 원인 효과를 증명한 것은 아니다. 결제 합계는 이익이나 순매출과도 다르다.

⑦ 이번 단계에서 배운 점

숫자만 맞는 것보다 대상·기간·단위·분모를 함께 설명하는 것이 중요하다. 재구매 비율의 분모는 주문 수가 아니라 고객 수다.

STEP 06 EDA 해석

그래프에서 질문과 실행 제안으로 연결한 과정

① 실습 목적

그래프를 보고 "그래서 무엇을 할 수 있을까"라는 질문으로 연결한다.

② 왜 이 작업이 필요한가

매출 그래프 한 장만으로는 성장이 주문 수 때문인지 주문 금액 때문인지 알기 어렵다. 고객·지역·시점·상품을 나눠 봐야 구체적인 제안을 만들 수 있다.

③ 사용한 기술과 코드

groupby는 같은 고객이나 같은 월끼리 묶는 함수다. 집계(Aggregation)는 묶인 값의 개수·합계·평균을 구하는 일이다. Olist에서는 고객별 주문 수와 월별 매출을 계산하는 데 사용한다.

```
# 해설: 고객별 주문 수를 세어 재구매 비율을 본다.
# 인사이트 1. 고객별 주문 횟수
# customer_unique_id : 실제 한 사람을 구분하는 번호

orders_per_customer = base.groupby("customer_unique_id")["order_id"].nunique()

one_time_rate = (orders_per_customer == 1).mean()   # True(1)의 평균 = 비율

print(f"고객 수              : {len(orders_per_customer):,}명")
```

핵심 코드 발췌 · 앞뒤 코드 생략
STEP1_Model_Training/05_eda.ipynb · 셀 22 · 일부 발췌 · 해설 주석 추가, 계산 로직 유지

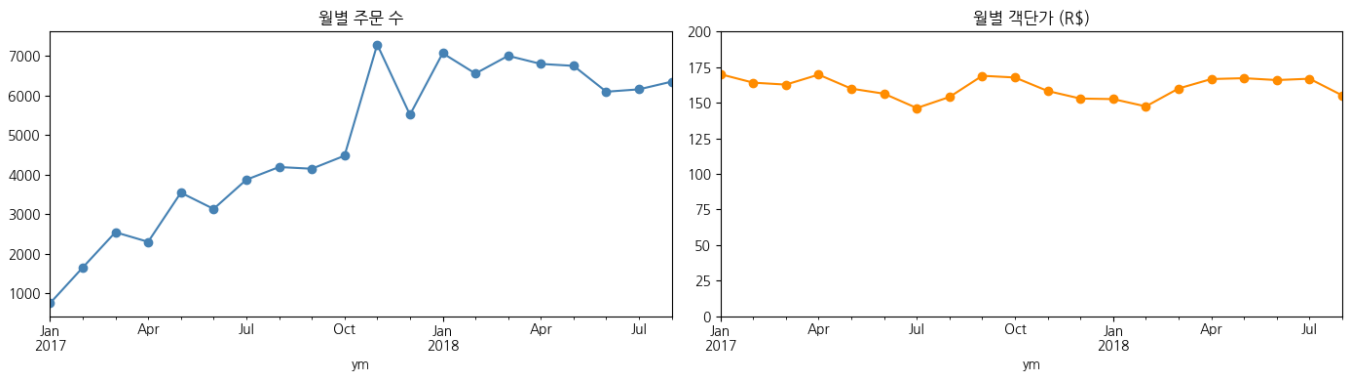
④ 실습 과정

1. 05_eda.ipynb의 고객·지역·결제·객단가 분석을 순서대로 읽었다.
2. 셀 34의 연도별 비교 그래프에서 같은 관측 기간을 비교하는지 확인했다.
3. 관찰→실행 제안→추가 측정 지표로 정리했다.

⑤ 실행 결과

관찰 또는 저장 결과	제안	추가 확인할 지표
재구매 고객 약 3.0%	두 번째 구매 유도 장치 검토	쿠폰 사용·재구매 전환
지역 집중	물류·마케팅 지역 우선순위 검토	지역별 배송기간·성과
지연 주문 리뷰가 낮음	지연 사전 안내와 원인 점검	동일 조건 비교·개선 후 변화
고객 구매 이력 희소	고객 협업 추천보다 상품 특징 추천	추천 클릭과 실제 구매

노트북 저장 출력



05_eda 셀 34 저장 그래프. 연도 비교에서는 관측된 기간을 맞춰야 한다.

⑥ 결과 해석

재구매 유도와 지연 사전 안내는 분석에 따른 제안이다. 실제로 캠페인을 시행해 매출이나 리뷰가 좋아졌다는 결과는 없다.

⑦ 이번 단계에서 배운 점

분석에서 관찰한 사실과 앞으로 해 볼 가설을 구분해야 한다. 좋은 제안은 나중에 무엇을 측정할지도 함께 제시한다.

STEP 07 추천 모델

유사 상품 추천 모델을 만든 과정

① 실습 목적

가격·크기·무게 등이 비슷한 상품 5개를 추천한다.

② 왜 이 작업이 필요한가

이 데이터는 고객의 반복 구매 이력이 적다. "비슷한 사람이 산 상품"을 찾는 대신 "지금 보는 상품과 특징이 가까운 상품"을 찾는 방법을 선택했다.

③ 사용한 기술과 코드

머신러닝(Machine Learning)은 데이터의 패턴을 이용하는 방법이다. 특징(Feature)은 판단에 사용하는 정보로 여기서는 가격·무게·크기·사진 수·평점이다. K 최근접 이웃(K Nearest Neighbors, KNN)은 특징을 숫자 공간에 놓고 가까운 이웃을 찾는다.

```
# 해설: 단위를 맞춘 뒤 카테고리 안에서 이웃을 찾는다.
# 셀 11. 특징 전처리 + 카테고리별 KNN 학습

features = ["price", "weight_g", "length_cm", "height_cm", "width_cm", "photos_qty",
"avg_review_score"]

X = np.log1p(data[features])                # ① 로그 변환
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)          # ② 스케일링 (평균 0, 표준편차 1)

# ③ 카테고리마다 따로 KNN을 학습합니다. (카테고리 = 필터)
K = 5
knn_by_category = {}
for cat, idx in data.groupby("category").groups.items():
    idx = np.array(idx)
    nn = NearestNeighbors(n_neighbors=K + 1)     # +1 : 자기 자신이 가장 가까우므로 하나 더
    nn.fit(X_scaled[idx])
    knn_by_category[cat] = (nn, idx)
```

핵심 코드 발췌 · 앞뒤 코드 생략
STEP1_Model_Training/06_recommendation.ipynb · 셀 22 · 일부 발췌 · 해설 주석 추가, 계산 로직 유지

④ 실습 과정

1. 06_recommendation.ipynb에서 상품별 특징을 준비했다.
2. 셀 22에서 로그 변환과 StandardScaler로 단위를 맞추고 카테고리별 KNN을 구성했다.
3. 셀 26의 저장 비교와 셀 30의 저장 모델 구성을 확인했다. 이번에 재학습하지 않았다.

⑤ 실행 결과

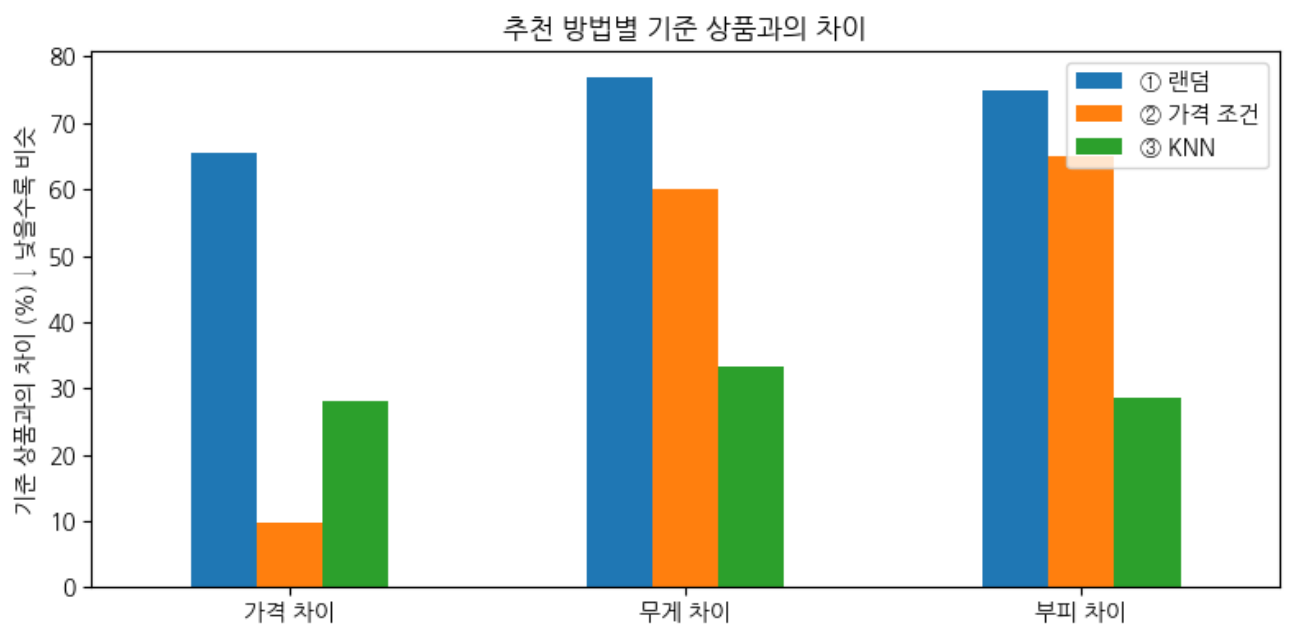
```
가격 조건으로 후보 5개를 채우지 못한 상품 : 150개 / 2,498개 (6.0%)

        가격 차이 무게 차이 부피 차이   평균
방법
① 랜덤       65%   77%   75%  72%
② 가격 조건   10%   60%   65%  45%
③ KNN      28%   33%   29%  30%
```

STEP1_Model_Training/06_recommendation.ipynb · 셀 26 저장 출력 · 이번 재학습 아님

저장 출력 기준 추천 대상은 2,498개 상품이다. 세 방법의 가격·무게·부피 상대 차이 중앙값을 평균하면 랜덤 약 72%, 가격 조건 약 45%, KNN 약 30%다. 이는 추천 상품의 특징 유사도 비교이며 클릭률·정확도·매출 증가율이 아니다.

노트북 저장 출력



06_recommendation 셀 27의 저장 그래프. 특징 차이 비교이며 추천 정확도가 아니다.

⑥ 결과 해석

72%·45%·30%는 가격·무게·부피의 상대 차이를 요약한 값이다. 낮을수록 이 특징들이 더 비슷하다는 뜻이며 추천 정확도·구매 확률·매출 개선율은 아니다.

⑦ 이번 단계에서 배운 점

목적에 맞는 입력과 평가를 골라야 한다. 비슷한 특징을 찾는 성과와 고객이 실제로 좋아하는 성과는 따로 측정해야 한다.

STEP 08 추천 결과

추천 결과를 서비스 화면에서 확인한 내용

① 실습 목적

저장한 추천 모델이 화면에서 어떤 상품을 보여 주는지 확인한다.

② 왜 이 작업이 필요한가

학습 노트북의 결과가 좋아도 사용자가 상품을 고르는 화면과 연결되지 않으면 활용하기 어렵다. 입력부터 추천 목록까지 흐름을 확인해야 한다.

③ 사용한 기술과 코드

추론(Inference)은 저장한 모델로 입력에 대한 결과를 구하는 일이다. scaler는 학습 때 정한 단위 변환을 다시 적용한다. kneighbors는 가까운 상품의 위치를 찾고 자기 자신을 제외해 5개를 남긴다.

```
# 해설: 학습 때의 변환을 적용하고 자기 자신은 추천에서 뺀다.
x = art["scaler"].transform(
    np.log1p(prods.loc[[i], art["features"]]))
nn, cat_idx = art["knn_by_category"][prods.at[i, "category"]]
_, pos = nn.kneighbors(x, n_neighbors=art["k"] + 1)
rec = cat_idx[pos[0]]
return prods.loc[rec[rec != i][:art["k"]]]
```

핵심 코드 발췌 · 앞뒤 코드 생략
utils/models.py · recommend 함수 일부 · 해설 주석 추가, 계산 로직 유지

④ 실습 과정

1. views/recommend.py에서 카테고리와 기준 상품을 고른다.
2. utils/models.py의 recommend 함수가 저장 모델과 같은 전처리를 적용한다.
3. 문서화 중 실제 로컬 화면에서 기준 상품 2548af3e와 추천 5개를 확인했다.

⑤ 실행 결과

화면 순서: 카테고리 선택 → 기준 상품 선택 → 기준 상품 표 확인 → 추천 5개 비교. 기준 상품은 R$10, 900g이며 추천 결과에는 같은 카테고리 상품이 표시됐다. 가격은 크게 다를 수 있어 다차원 특징 추천의 한계도 확인할 수 있다.

문서화 중 실제 화면 확인



문서화 중 실제 원본 로컬 앱에서 확인한 상품 추천 화면. 가구·장식의 기준 상품 2548af3e와 추천 5개.

⑥ 결과 해석

같은 가구·장식 안에서도 가격은 크게 다를 수 있다. 여러 특징을 함께 비교하는 추천이므로 가격만 비슷하도록 보장하는 기능은 아니다.

⑦ 이번 단계에서 배운 점

모델은 전처리 도구·특징 목록과 함께 보관해야 한다. 추천 목록의 의미와 한계도 화면 설명에 적어야 사용자가 오해하지 않는다.

STEP 09 감성 분석 ML

리뷰 문장을 긍정과 부정으로 분류한 과정

① 실습 목적

포르투갈어 리뷰 문장을 긍정과 부정으로 나눈다.

② 왜 이 작업이 필요한가

리뷰가 많으면 사람이 모두 읽기 어렵다. 우선 살펴볼 불만 리뷰를 찾는 데 도움이 되지만, 별점과 문장 내용이 항상 같은 감정은 아니다.

③ 사용한 기술과 코드

단어 빈도와 문서 빈도를 반영하는 TF-IDF(Term Frequency–Inverse Document Frequency)는 문장을 단어별 숫자로 바꾼다. 로지스틱 회귀(Logistic Regression)는 이 숫자로 긍정 클래스의 확률 점수를 추정한다. fit은 학습, transform은 학습한 기준의 적용이다.

```
# 해설: 문장을 단어 점수표로 바꾸고 학습과 시험을 분리한다.
# 셀 4. 데이터 전처리 + 라벨 만들기

review_df = review_df.dropna()                                  # 리뷰 글이 없는 행 제거
review_df = review_df[review_df["review_score"] != 3]       # 중립(3점) 제거

# 4~5점 → 긍정(1),  1~2점 → 부정(0)
review_df["label"] = (review_df["review_score"] >= 4).astype(int)

print(f"분석할 리뷰 수: {len(review_df):,}")
print(review_df["label"].value_counts())
#   긍정이 부정보다 약 2.5배 많습니다. → 불균형 데이터
```

핵심 코드 발췌 · 앞뒤 코드 생략
STEP1_Model_Training/07_1_sentiment_ml.ipynb · 셀 7 · 일부 발췌 · 해설 주석 추가, 계산 로직 유지

```
# 해설: 문장을 단어 점수표로 바꾸고 학습과 시험을 분리한다.
# 셀 6. TF-IDF 벡터 만들기
# max_features=10000 : 자주 나오는 단어 10,000개만 사용 (PART 7-2 딥러닝과 같은 크기)
#    fit은 train으로만! test는 transform만 합니다. (test의 단어 정보를 미리 보면 누수)

tfidf = TfidfVectorizer(max_features=10000)
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf  = tfidf.transform(X_test)
```

핵심 코드 발췌 · 앞뒤 코드 생략
STEP1_Model_Training/07_1_sentiment_ml.ipynb · 셀 10 · 일부 발췌 · 해설 주석 추가, 계산 로직 유지

④ 실습 과정

1. 07_1_sentiment_ml.ipynb 셀 7에서 글 없는 리뷰와 3점을 제외했다.
2. 셀 8에서 학습·시험을 나누고 셀 10에서 학습 자료에만 단어 사전을 맞췄다.
3. 셀 13과 21의 기본·개선 평가를 구분해 읽었다. 재학습은 하지 않았다.

⑤ 실행 결과

모델	시험 정확도	출처 상태
전부 긍정 기준선	0.710	07_1 셀 13 저장 출력
TF IDF 기본 로지스틱 회귀	0.917	같은 시험 데이터의 저장 출력
bigram과 악센트 통일 개선	0.926	07_1 셀 21 저장 출력

저장 출력: 리뷰 37,139개, 긍정 26,380개, 부정 10,759개다. 학습 29,711개와 시험 7,428개를 random_state=42, stratify 조건으로 분리했다. TF IDF의 fit은 학습 데이터에만 적용하고 시험 데이터에는 transform만 적용했다.

실제 배포 저장 셀은 기본 tfidf와 model을 저장한다. 개선 모델 0.926을 그대로 배포 모델의 성능이라고 표기하지 않았다. 613개의 오분류에는 별점과 내용의 차이, 부정 표현의 문맥 문제가 포함됐다.

⑥ 결과 해석

기본 0.917과 개선 0.926은 서로 다른 특징 설정의 시험 정확도다. 배포 저장 셀은 기본 모델을 저장하므로 0.926을 배포 모델의 성능이라고 쓰지 않는다.

⑦ 이번 단계에서 배운 점

모델을 바꾸기 전에 문장 표현을 숫자로 바꾸는 방법부터 개선할 수 있다. 시험 자료를 미리 학습에 사용하면 평가가 부풀려질 수 있다.

STEP 10 감성 모델 비교

딥러닝과 BERT를 비교하며 확인한 한계

① 실습 목적

같은 리뷰 문제를 딥러닝과 BERT로 풀었을 때의 차이를 비교한다.

② 왜 이 작업이 필요한가

큰 모델이라는 이유만으로 좋은 결과가 보장되지 않는다. 같은 언어·정답 기준·시험 표본인지 확인해야 공정한 비교가 된다.

③ 사용한 기술과 코드

딥러닝(Deep Learning)은 여러 층의 신경망으로 패턴을 학습하는 방법이다. BERT(Bidirectional Encoder Representations from Transformers)는 문장 앞뒤 문맥을 이용하는 언어 모델이다. 임베딩(Embedding)은 단어나 문장의 특징을 숫자 목록으로 바꾸는 표현이다.

```
# 해설: 검증 손실이 더 좋아지지 않으면 멈춘다.
# 셀 12. 모델 학습 (약 30초~1분)

early_stop = EarlyStopping(
    monitor="val_loss",           # 검증 손실을 지켜봄
    patience=2,                   # 2번 연속 좋아지지 않으면 멈춤
    restore_best_weights=True,    # 가장 좋았던 시점의 가중치로 되돌림
)

history = model.fit(
    X_train_pad, y_train,
    epochs=10,            # 최대 10번 반복 (보통 그 전에 EarlyStopping이 멈춤)
    batch_size=64,        # 한 번에 64개씩 처리, 64문장마다 한 번씩 파라미터를 업데이트
    validation_data=(X_val_pad, y_val),  # 미리 떼어 둔 검증 데이터로 매 epoch 성능 확인
    class_weight=class_weight,           # 부정 리뷰를 더 중요하게 학습 (불균형 보정)
    callbacks=[early_stop],              # 학습 도중 멈출 조건
    verbose=2             # epoch마다 결과를 한 줄씩 출력
```

핵심 코드 발췌 · 앞뒤 코드 생략
STEP1_Model_Training/07_2_sentiment_dl.ipynb · 셀 23 · 일부 발췌 · 해설 주석 추가, 계산 로직 유지

④ 실습 과정

1. 07_2_sentiment_dl.ipynb에서 토큰 번호와 길이 50의 입력을 준비했다.
2. 셀 23에서 검증 손실을 보고 학습을 멈추는 EarlyStopping을 적용한 저장 기록을 확인했다.
3. 07_3_sentiment_bert.ipynb 셀 18의 같은 500개 비교와 전체 시험 결과를 구분했다.

⑤ 실행 결과

비교 항목	저장 결과	해석
Embedding 평균 모델	시험 정확도 0.9235	같은 이진 시험 집합
학습 중단	7 epoch 실행, 최적 5 epoch	검증 손실이 좋은 가중치 복원
BERT 중립 포함	500개에서 정확도 0.730	3분류 결과의 neutral을 이진 평가에서 오답 처리
BERT 중립 제외	0.787	평가 분모가 달라 직접 우열 판단 주의
같은 500개 ML·DL	0.918 / 0.922	07_3 셀 18의 저장 비교

07_2는 리뷰를 토큰 번호로 바꾸고 길이 50으로 맞춘 뒤 Embedding과 GlobalAveragePooling1D 기반 모델 및 Bi LSTM을 비교한다. 학습·검증·시험을 구분하고 검증 손실을 보고 EarlyStopping으로 멈춘다.

⑥ 결과 해석

BERT의 0.730은 500개에서 neutral을 오답으로 센 값이다. 중립을 제외한 0.787은 분모도 달라진다. 전체 시험 7,428개의 DL 결과와 바로 순위를 매길 수 없다.

⑦ 이번 단계에서 배운 점

성능 숫자 옆에는 평가 대상과 정답 기준이 있어야 한다. 모델 크기뿐 아니라 비용·언어·전처리·실제 사용 목적을 함께 고려한다.

STEP 11 판매자 군집

판매자를 비슷한 유형으로 묶은 과정

① 실습 목적

판매자가 보이는 활동 패턴에 따라 비슷한 집단으로 나눈다.

② 왜 이 작업이 필요한가

판매가 활발한 판매자와 오래 쉬는 판매자에게 같은 지원을 할 필요는 없다. 집단별 특징을 이해하면 운영 방향을 다르게 생각할 수 있다.

③ 사용한 기술과 코드

K 평균 군집화(K-Means Clustering)는 비슷한 숫자 특징을 가진 대상을 k개 집단으로 묶는 방법이다. 정답을 미리 주지 않는다. 실루엣 점수(Silhouette Score)는 집단이 얼마나 잘 구분되는지 보는 참고 지표다.

```
# 해설: 판매자 특징을 네 집단으로 묶는다.
# 셀 10. K-Means 학습

K = 4                                                    # 셀 9에서 실루엣 점수가 가장 높았던 k
kmeans = KMeans(n_clusters=K, n_init=20, random_state=42)
prof["segment_id"] = kmeans.fit_predict(X_scaled)        # 학습 + 판매자마다 군집 번호(0~3) 붙이기

# 군집별 특징 (중앙값 : 치우친 값에 덜 흔들림)
summary = prof.groupby("segment_id").agg(
    판매자수=("seller_id", "size"),
    주문건수=("frequency", "median"),
    판매액=("monetary", "median"),
    마지막판매후일수=("recency_days", "median"),
```

핵심 코드 발췌 · 앞뒤 코드 생략
STEP1_Model_Training/08_seller_segment.ipynb · 셀 19 · 일부 발췌 · 해설 주석 추가, 계산 로직 유지

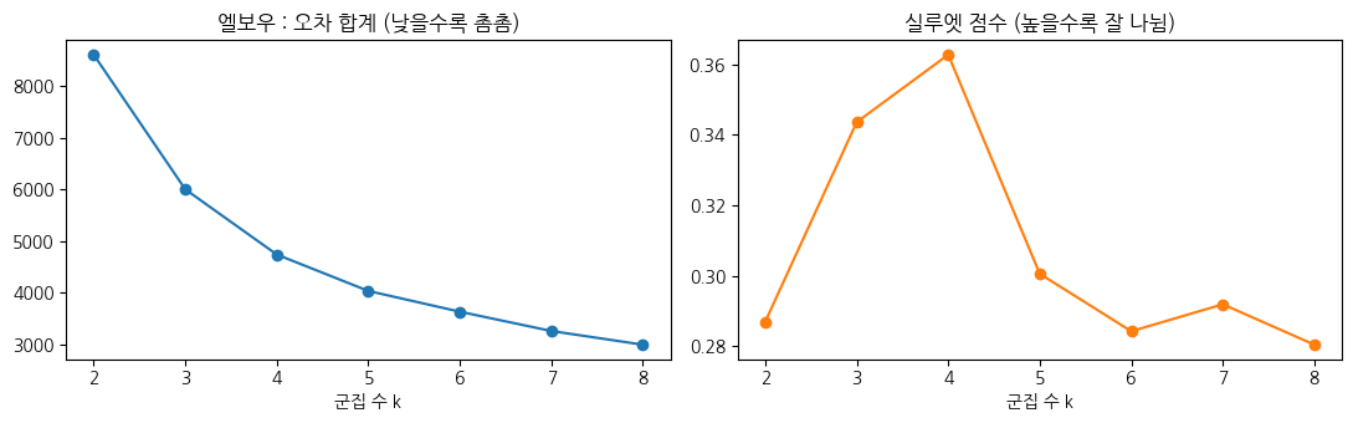
④ 실습 과정

1. 08_seller_segment.ipynb에서 2018-06-01 기준 프로필을 만들었다.
2. 셀 13에서 빈 리뷰 평점을 채우고 로그·스케일링을 확인했다.
3. 셀 17의 k 비교, 셀 21의 유형 이름, 셀 28의 저장 기록을 읽었다. 이번 새 학습은 아니다.

⑤ 실행 결과

유형	판매자 수	특성 해석
핵심 우량	876	주문과 판매액이 크고 최근 활동
신규 성장	947	시작한 지 짧고 최근 판매
휴면	461	마지막 판매 후 오랜 기간 경과
저평점 위험	218	리뷰 평점이 낮은 집단

노트북 저장 출력

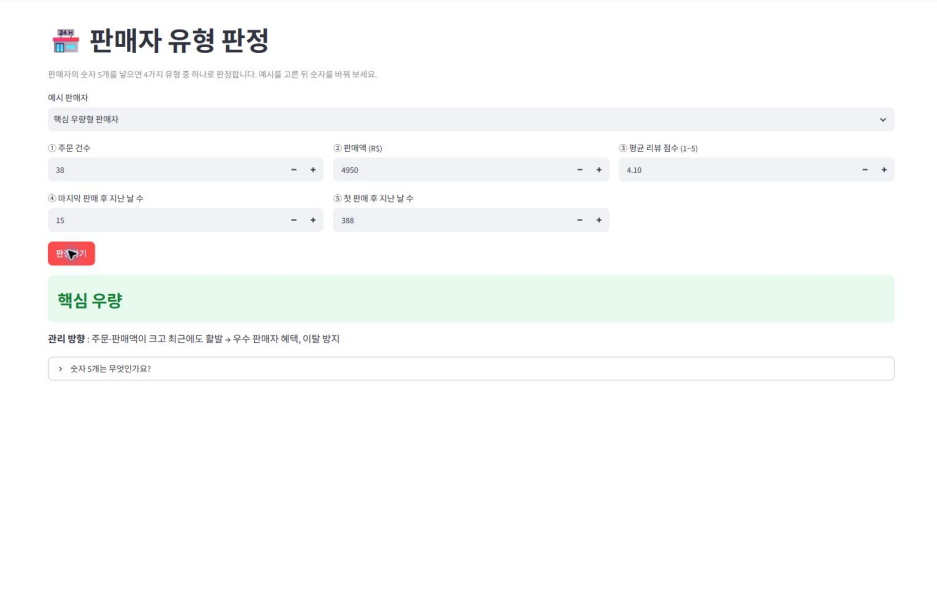


08_seller_segment 셀 17 저장 그래프. k 후보 중 실루엣 점수가 가장 높은 값은 4로 기록돼 있다.

목적: 주문 빈도, 판매액, 최근성, 활동기간, 평균 평점이 비슷한 판매자끼리 묶고 유형에 맞는 운영 방향을 해석한다. 2018-06-01 기준 2,502명을 대상으로 로그 변환·스케일링 후 K 평균 군집화를 수행했다.

리뷰가 없는 53명의 평점은 중앙값 4.14로 채웠다. 최종 이름은 군집의 요약 특성을 보고 붙인 해석이다. k=4는 수업 데이터와 특징 조건에서의 선택이며 보편적인 판매자 유형 개수를 뜻하지 않는다.

문서화 중 실제 화면 확인



추가 확인 화면. 예시 판매자의 5개 수치를 입력한 결과이며 전체 모델 성능 검증은 아니다.

⑥ 결과 해석

2,502명에서 k=4를 선택해 이름을 붙였다. 핵심 우량·신규 성장·휴면·저평점 위험은 집단의 특성을 설명하는 이름이며 모든 판매자에게 적용되는 객관적 등급은 아니다.

⑦ 이번 단계에서 배운 점

같은 군집 번호라도 다시 학습하면 의미가 달라질 수 있다. 유형 이름만 보관하지 말고 모델·전처리·해석 기준을 함께 남겨야 한다.

STEP 12 이탈 예측

미래 주문이 없는 판매자를 예측한 과정

① 실습 목적

최근 활동한 판매자 중 다음 3개월에 주문이 없을 대상을 예측한다.

② 왜 이 작업이 필요한가

판매자가 떠난 뒤에만 대응하면 늦을 수 있다. 다만 "다음 기간 주문 없음"은 수업용 정의이며 실제 폐업이나 계약 해지와 같지 않다.

③ 사용한 기술과 코드

분류(Classification)는 미리 정한 정답 범주를 예측하는 작업이다. 데이터 누수(Data Leakage)는 예측 시점에 알 수 없던 미래 정보를 입력에 섞는 실수다. 이 실습은 기준일 뒤에 생긴 배송·리뷰 정보가 입력에 들어가지 않도록 제한한다.

```
# 해설: 과거 특징과 미래 정답을 나눈 뒤 시험 자료를 떼어 둔다.
# 셀 7. 특징 목록 + test 분리 (시험지는 처음에 떼어 두고 손대지 않습니다)

features = [
    "recency_days", "tenure_days", "active_months",
    "log_frequency", "log_monetary", "log_avg_order_value",
    "avg_delivery_days", "late_rate", "ship_late_rate", "avg_review_score",
"low_review_rate",
    "log_orders_last_3m", "log_orders_prev_3m", "log_order_trend",
]
# 주문 수·금액은 로그 변환한 값을 씁니다. (make_churn_features 안에서 계산)
X, y = df[features], df["churn"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)   # 이탈 비율을 train/test에 똑같이
print(f"train {len(X_train):,}명 (이탈 {y_train.mean():.1%})  /  test {len(X_test):,}명 (이탈
{y_test.mean():.1%})")
print("결측이 있는 특징 :", X_train.columns[X_train.isna().any()].tolist())
#   배송·리뷰가 아직 없는 판매자는 빈 값입니다. Pipeline 안에서 train의 중앙값으로 채웁니다.
```

핵심 코드 발췌 · 앞뒤 코드 생략
STEP1_Model_Training/09_seller_churn.ipynb · 셀 11 · 일부 발췌 · 해설 주석 추가, 계산 로직 유지

④ 실습 과정

1. 09_seller_churn.ipynb에서 과거 정보로 판매자 특징을 만들었다.
2. 셀 9에서 미래 3개월 주문으로 정답을 만들고 셀 11에서 학습·시험을 분리했다.
3. 최근 30일 규칙과 모델을 비교하는 저장 기록을 읽었다.

⑤ 실행 결과

설계	확인 내용
기준일	2018-06-01
대상	2018년 3~5월 활동 1,567명
정답	이후 6~8월 유효 주문 0건: 377명, 24.1%
분리	학습 1,253명 / 시험 314명
특징	최근성과 활동기간, 주문과 금액, 배송과 리뷰, 최근 주문 추세 등 14개
누수 방지	기준일 이전에 알려진 배송과 리뷰만 사용

목적: 최근 활동한 판매자 중 이후 3개월 동안 주문이 없는 대상을 예측한다. 실제 폐업 여부가 아니라 수업에서 정한 주문 활동 기준의 이탈이다.

먼저 최근 30일 판매 없음 규칙을 만들고, 학습 집합 내부 5회 교차검증으로 모델을 비교했다. 시험 데이터는 선택 과정에 사용하지 않고 최종 평가에 사용하도록 작성돼 있다.

⑥ 결과 해석

학습 대상 1,567명 중 377명이 수업 정의의 이탈이다. 14개 특징은 판매 활동·배송·리뷰·추세를 나타내며 미래 결과 자체를 입력하는 열이 아니다.

⑦ 이번 단계에서 배운 점

예측하려는 사건을 먼저 분명히 정의해야 한다. 과거 입력과 미래 정답의 시간 경계를 지키는 것이 모델 선택만큼 중요하다.

STEP 13 모델 평가

이탈 모델의 평가를 읽고 선택한 근거

① 실습 목적

이탈 모델이 누구를 잘 찾고 누구에게 잘못 경보하는지 읽는다.

② 왜 이 작업이 필요한가

모두 유지라고 예측해도 다수 집단 때문에 정확도가 높아 보일 수 있다. 실제 이탈자를 놓치는 비용과 잘못 연락하는 비용을 함께 봐야 한다.

③ 사용한 기술과 코드

정밀도(Precision)는 경보한 사람 중 실제 이탈자의 비율이다. 재현율(Recall)은 실제 이탈자 중 찾아낸 비율이고, F1 점수(F1 Score)는 둘의 균형을 한 숫자로 요약한다. 교차검증(Cross-validation)은 학습 자료를 여러 번 나눠 성능의 흔들림을 살피는 방법이다.

```
# 해설: 시험 자료에서 예측이 얼마나 맞았는지 확인한다.
# 셀 13. 로지스틱 회귀 상세 평가

lr_model = final["로지스틱 회귀 + balanced"]
pred = lr_model.predict(X_test)
print(classification_report(y_test, pred, target_names=["유지(0)", "이탈(1)"], digits=3))
cm = confusion_matrix(y_test, pred)
print("=== Confusion Matrix ===")
print(pd.DataFrame(cm, index=["실제 유지", "실제 이탈"], columns=["예측 유지", "예측 이탈"]))
```

핵심 코드 발췌 · 앞뒤 코드 생략
STEP1_Model_Training/09_seller_churn.ipynb · 셀 24 · 일부 발췌 · 해설 주석 추가, 계산 로직 유지

④ 실습 과정

1. 09_seller_churn.ipynb 셀 17의 5회 교차검증을 비교했다.
2. 셀 22에서 따로 남겨 둔 시험 결과를 읽었다.
3. 셀 24의 혼동행렬에서 이탈 76명 중 57명을 찾고 19명을 놓쳤음을 확인했다. 모두 저장 출력이다.

⑤ 실행 결과

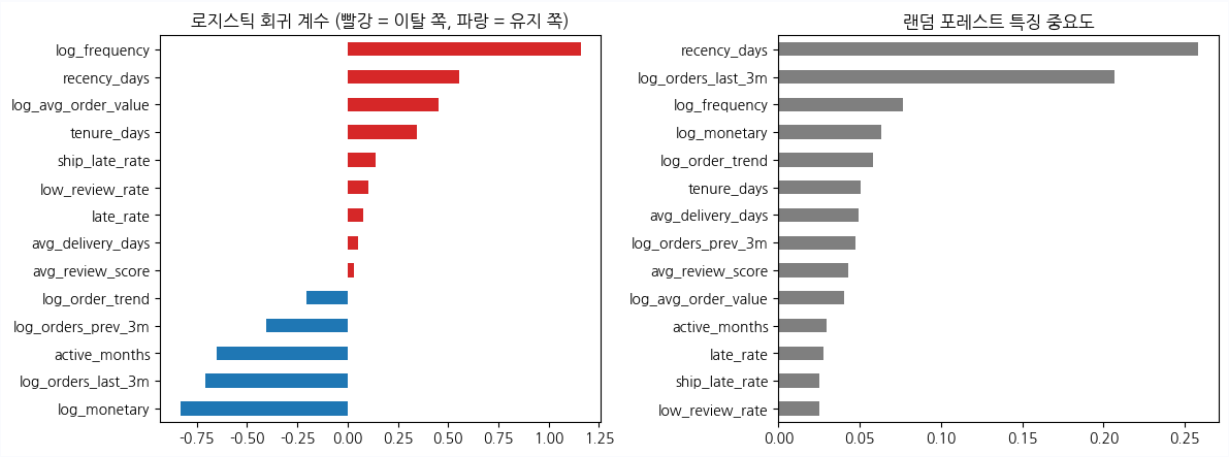
5회 교차검증	정밀도	재현율	F1	AUC
30일 규칙	0.49	0.58	0.53	0.77
로지스틱 회귀	0.59	0.32	0.40	0.79
로지스틱 balanced	0.47	0.75	0.58	0.80
랜덤 포레스트 balanced	0.48	0.67	0.55	0.80

시험 데이터 314명	예측 유지	예측 이탈
실제 유지 238명	164	74
실제 이탈 76명	19	57

출처는 09_seller_churn 셀 17 저장 출력이다. balanced 로지스틱 회귀는 5회 모두 규칙보다 F1이 높게 기록됐다. 평균뿐 아니라 흔들림을 확인한 뒤 배포 모델로 선택하는 흐름이다.

셀 24 저장 출력에서 이탈 정밀도는 0.435, 재현율은 0.750, F1은 0.551이다. 실제 이탈 76명 중 57명을 찾았지만 유지 판매자 74명에게도 경보를 냈다. 연락 비용과 놓치는 비용의 균형을 설명해야 한다.

노트북 저장 출력



09_seller_churn 셀 27에 저장된 특징 해석 그래프. 모델의 연관 패턴이며 원인 증명은 아니다.

⑥ 결과 해석

balanced 로지스틱 회귀의 시험 정밀도 0.435, 재현율 0.750, F1 0.551이다. 이탈을 많이 찾는 대신 유지 판매자 74명에게도 경보를 냈다. 교차검증 F1 0.58과 시험 F1 0.55는 다른 평가다.

⑦ 이번 단계에서 배운 점

평가 숫자는 어떤 오판을 했는지로 풀어서 설명해야 한다. 전체 데이터로 다시 학습해 저장한 배포 모델을 독립 시험처럼 평가하면 안 된다.

STEP 14 전략 적용

모델을 운영 우선순위로 연결한 과정

① 실습 목적

이탈 확률과 최근 판매액을 함께 보며 연락 우선순위를 정한다.

② 왜 이 작업이 필요한가

확률만 보면 판매 규모가 작은 판매자가 먼저 올 수 있고 판매액만 보면 위험이 낮은 판매자가 먼저 올 수 있다. 두 요소를 함께 보는 점수를 만든다.

③ 사용한 기술과 코드

predict_proba는 클래스별 확률 점수를 돌려준다. expected_loss는 그 점수에 최근 3개월 판매액을 곱한 값이다. 이름에 loss가 있어도 실제 손실을 측정한 금액은 아니다.

```
# 해설: 위험 점수와 최근 판매액을 함께 본다.
table["churn_prob"] = churn["model"].predict_proba(
    table[churn["features"]])[:, 1]
table["risk"] = table["churn_prob"].map(risk_label)
table["expected_loss"] = (
    table["churn_prob"] * table["sales_last_3m"])
```

핵심 코드 발췌 · 앞뒤 코드 생략
utils/models.py · score_sellers 일부 · 해설 주석 추가, 계산 로직 유지

④ 실습 과정

1. 10_business_strategy.ipynb 셀 14~16의 최신 프로필과 우선순위 기록을 확인했다.
2. utils/models.py의 score_sellers를 따라 저장 모델과 판매자 표를 다시 계산했다.
3. 실제 이탈 위험 화면의 대상·위험 인원과 대조했다.

⑤ 실행 결과

10_business_strategy는 학습 기준일 뒤의 최신 프로필을 2018-08-23 기준으로 만들고 판매자 표에 이탈 확률과 유형을 붙인다. 현재 모델 파일과 seller_table.csv를 다시 읽어 대상 1,817명 중 확률 0.5 이상 637명을 확인했다.

expected_loss는 이탈 확률×최근 3개월 판매액의 우선순위 점수다. 실제 미래 손실의 검증된 예측이나 회계상 손실이 아니다. 첫 판매자의 확률은 약 92.4%로 표시되지만 그 판매자가 실제로 이탈했다는 미래 정답은 확인하지 않았다.

문서화 중 실제 화면 확인



판매자 이탈 위험

2018-08-23 기준, 최근 3개월 안에 판매가 있었던 판매자가 앞으로 3개월 동안 주문을 못 받을 확률입니다.

대상 판매자 1,817명 | 이탈 위험 (확률 0.5 이상) 637명 | 예상 손실 합계 R$ 510,386

먼저 챙길 판매자 TOP 10

유형: 전체 / 핵심 우량 / 신규 성장 / 휴면 / 저평점 위험

판매자	유형	이탈 확률	위험 등급	최근 3개월 주문	마지막 판매 후 일수	최근 3개월 판매액(R$)	예상 손실(R$)
b37c4c02	핵심 우량	0.92	매우 높음	1	38	7160	6618
40db9e9a	핵심 우량	0.33	보통	9	5	17728	5849
80ceebb4	신규 성장	0.86	매우 높음	1	28	6729	5773
8fb79102	신규 성장	0.46	보통	5	31	8120	3734
2bf6a2c1	핵심 우량	0.25	낮음	8	9	13894	3502
52f976b1	핵심 우량	0.58	높음	4	30	5030	2941
2eb70248	핵심 우량	0.17	낮음	80	34	17167	2916
b839e417	핵심 우량	0.11	낮음	22	2	27118	2865
c4d51195	핵심 우량	0.76	매우 높음	6	78	3704	2822
87e24fc7	신규 성장	0.41	보통	6	8	6824	2795

판매자 한 명 자세히 보기

판매자 선택 (위 표 순서): b37c4c02

이탈 확률 | 유형 | 최근 3개월 주문 | 예상 손실

문서화 중 원본 로컬 앱에서 확인한 이탈 위험 화면. 대상 1,817명, 위험 637명, 우선순위 점수 합계 약 R$510,386.

⑥ 결과 해석

2018-08-23 기준 대상 1,817명, 확률 0.5 이상 637명을 확인했다. 화면의 약 R$510,386은 우선순위 점수 합계이며 검증된 미래 손실액이 아니다.

⑦ 이번 단계에서 배운 점

예측을 실제 업무로 연결하려면 확률뿐 아니라 영향과 대응 비용도 생각해야 한다. 실험하지 않은 캠페인의 매출 개선 효과는 주장하지 않는다.

STEP 15 모델 서빙

학습한 모델을 Gradio 입력 화면으로 연결한 과정

① 실습 목적

학습한 감성 모델에 문장을 넣고 ML과 DL의 결과를 비교하는 창구를 만든다.

② 왜 이 작업이 필요한가

모델 함수를 호출하는 코드를 모르는 사람도 문장을 넣어 결과를 볼 수 있어야 한다. 단순 입력 화면으로 연결하는 것이 서빙의 첫 단계다.

③ 사용한 기술과 코드

그라디오(Gradio)는 예측 함수를 입력 칸과 출력 화면으로 연결하는 도구다. 모델 서빙(Model Serving)은 저장 모델이 사용자의 요청을 받아 결과를 돌려주는 과정이다.

```
# 해설: 문장을 각 모델의 입력 형식으로 바꾸어 예측한다.
# 셀 4. 예측 함수 (각 노트북 마지막 '저장한 파일로 예측해 보기' 셀과 같은 순서)
def predict_ml(text):
    return float(ml["model"].predict_proba(ml["tfidf"].transform([text]))[0, 1])   # 긍정
확률

def predict_dl(text):
    seq = tokenizer.texts_to_sequences([text])
    pad = pad_sequences(seq, maxlen=MAX_LEN, padding="post", truncating="post")
    return float(dl.predict(pad, verbose=0)[0][0])                                  # 긍정
 확률

text = "produto veio quebrado, péssimo"          # 상품이 깨져서 왔다, 최악
print(f"머신러닝 긍정 확률 : {predict_ml(text):.3f}")
print(f"딥러닝   긍정 확률 : {predict_dl(text):.3f}")
```

핵심 코드 발췌 · 앞뒤 코드 생략
STEP2_Model_Serving/01_sentiment_demo_gradio.ipynb · 셀 5 · 일부 발췌 · 해설 주석 추가, 계산 로직 유지

④ 실습 과정

1. STEP2의 01_sentiment_demo_gradio.ipynb에서 저장 모델과 tokenizer를 읽는다.
2. 셀 5의 예측 함수와 예문 결과를 확인했다.
3. 셀 7의 임시 Gradio 링크 생성 기록을 읽었다. 새 Gradio 화면은 실행하지 않았다.

⑤ 실행 결과

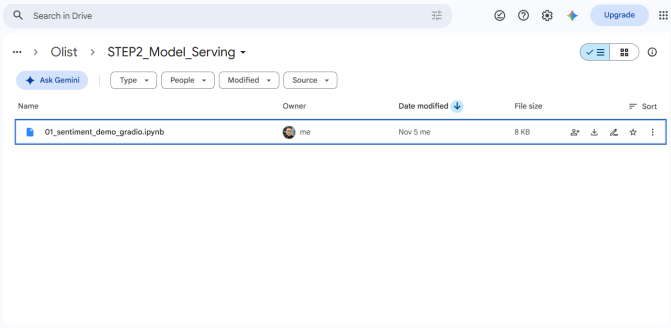
```
머신러닝 긍정 확률 : 0.006
딥러닝   긍정 확률 : 0.021
```

STEP2_Model_Serving/01_sentiment_demo_gradio.ipynb · 셀 5 저장 출력 · 이번 재학습 아님

저장 출력에는 같은 부정 예문에 대해 ML 긍정 확률 0.006, DL 긍정 확률 0.021이 기록돼 있다. 셀 7에는 Gradio 공개 임시 링크가 생성된 흔적도 있다. 이 과거 링크가 현재 계속 살아 있거나 영구 서비스된다고 주장하지 않는다.

이번 문서화에서는 Gradio 화면을 새로 실행하지 않았다. 기존 노트북 코드·저장 출력과 Drive 파일 위치를 연결해 실습 과정을 복원했다.

Drive 파일 목록



사용자 Google Drive의 STEP2 노트북 파일 확인 화면. 이것만으로 Gradio 실행 화면을 증명하지는 않는다.

⑥ 결과 해석

같은 부정 예문의 저장 출력은 ML 긍정 확률 0.006, DL 0.021이다. 이번 Streamlit에서 새로 확인한 0.6%와는 출처가 다르다. 과거 임시 링크의 현재 접속은 확인하지 않았다.

⑦ 이번 단계에서 배운 점

학습 결과를 화면으로 연결하려면 모델뿐 아니라 입력 전처리도 준비해야 한다. 과거 링크가 생성됐다는 사실과 지금 서비스가 동작한다는 사실을 구분한다.

STEP 16 배포 준비

로컬 대시보드용 파일을 준비한 과정

① 실습 목적

대시보드가 읽을 분석 표와 모델 파일을 정리한다.

② 왜 이 작업이 필요한가

큰 통합 CSV에서 매번 모든 계산을 하면 화면이 느려질 수 있다. 필요한 열과 분석 대상을 미리 정리해 화면과 계산을 분리한다.

③ 사용한 기술과 코드

파케이(Parquet)는 표 데이터를 열 중심으로 보관하는 파일 형식이다. to_parquet는 표를 저장하고 read_parquet는 다시 읽는다. 작은 파일을 만들기 위해 필요한 열만 고르는 처리도 함께 사용한다.

```
# 해설: 대시보드에 필요한 열만 추려 별도 파일로 저장한다.
                & orders["order_delivered_customer_date"].notna()
                & (orders["order_purchase_timestamp"] >= "2017-01-05")
                & (orders["order_purchase_timestamp"] < "2018-08-30")].copy()
orders["주문시각"] = orders["order_purchase_timestamp"]
orders["매출"] = orders["order_id"].map(pay)
orders["배송일수"] = (orders["order_delivered_customer_date"] -
orders["주문시각"]).dt.total_seconds() / 86400
orders["지연"] = orders["order_delivered_customer_date"] >
orders["order_estimated_delivery_date"]
orders = orders[["order_id", "customer_unique_id", "customer_state", "주문시각", "매출",
"배송일수", "지연", "review_score"]]

items = df.dropna(subset=["order_item_id"]).drop_duplicates(["order_id",
"order_item_id"])  # 상품 1개 = 1행
items = items[items["order_id"].isin(orders["order_id"])][["order_id",
"product_category_name_english", "price"]]

orders.to_parquet(f"{deploy_path}/data/orders.parquet", index=False)
items.to_parquet(f"{deploy_path}/data/items.parquet", index=False)
```

핵심 코드 발췌 · 앞뒤 코드 생략
STEP3_Local_Deploy/00_prepare_deploy_files.ipynb · 셀 4 · 일부 발췌 · 해설 주석 추가, 계산 로직 유지

④ 실습 과정

1. 00_prepare_deploy_files.ipynb 셀 4의 데이터 준비 규칙을 확인했다.
2. 원본 orders.parquet 읽기 실패를 기록했다.
3. 원본을 보존한 demo_app 복사본에 같은 CSV와 규칙으로 재생성하고 다시 읽어 지표를 대조했다.

⑤ 실행 결과

위치	필요 파일
data	orders.parquet, items.parquet, seller_table.csv
data RAG	olist_facts.csv, olist_model_guide.md, olist_seller_policy.pdf
model	part6_recommend, part7_1_sentiment_ml, part8_seller_segment, part9_seller_churn의 pkl
views와 utils	화면 코드와 데이터·모델·검색 함수

원본 orders.parquet를 읽을 때 Thrift 역직렬화 오류가 발생했다. items.parquet는 정상적으로 읽혔다. 파일 내부 손상 또는 형식 문제의 발생 원인은 확정하지 않았다. ZIP과 로컬 파일의 일치만으로 파일의 유효성을 보장할 수 없다는 사례다.

원본은 보존하고 output/olist/demo_app에 확인용 복사본을 만들었다. 같은 준비 노트북 셀의 규칙으로 CSV에서 Parquet를 다시 생성했다. 재생성 파일을 다시 읽어 96,203행과 결제 합계 R$15,374,496.52를 확인한 뒤 분석 화면을 열었다.

이 복구는 문서화 과정에서 수행한 추가 검증이다. 사용자가 과거 실습에서 겪거나 해결했다고 서술하지 않았다.

⑥ 결과 해석

재생성 파일에서 96,203행과 결제 합계 R$15,374,496.52를 확인했다. 원본 파일의 손상 또는 형식 문제가 언제 왜 생겼는지는 확정하지 않았다.

⑦ 이번 단계에서 배운 점

파일이 존재하거나 ZIP과 같다는 것만으로 정상 파일임을 보장하지 않는다. 복구 뒤에는 다시 읽기와 실제 화면까지 확인해야 한다.

STEP 17 웹 대시보드

Streamlit 분석과 모델 화면을 연결한 과정

① 실습 목적

분석 차트와 여러 모델 기능을 하나의 웹 대시보드로 묶는다.

② 왜 이 작업이 필요한가

노트북마다 다른 결과를 사용자가 찾아다니기보다 한 화면에서 메뉴로 고를 수 있으면 활용하기 쉽다.

③ 사용한 기술과 코드

스트림릿(Streamlit)은 Python의 표·차트·입력을 웹 화면으로 만드는 도구다. app.py는 메뉴를 등록하고 views는 각 화면, utils는 반복해서 쓰는 계산과 예측을 담당한다.

```
# 해설: 각 화면 파일을 메뉴에 연결한다.
st.Page("views/analysis.py", title="데이터 분석")
st.Page("views/sentiment.py", title="리뷰 감성 분석")
st.Page("views/recommend.py", title="상품 추천")
st.Page("views/segment.py", title="판매자 유형")
st.Page("views/churn.py", title="판매자 이탈 위험")
st.Page("views/qa.py", title="분석 Q&A")
```

핵심 코드 발췌 · 앞뒤 코드 생략
app.py · st.Page 등록문에서 icon 인자 생략 · 해설 주석 추가, 계산 로직 유지

④ 실습 과정

1. app.py의 7개 메뉴 등록을 확인했다.
2. 분석·감성·추천·유형·이탈 메뉴의 일부 동작을 실제 화면에서 확인했다.
3. 분석은 Parquet를 재생성한 별도 복사본에서 확인하고 화면 8장을 GIF로 묶었다.

⑤ 실행 결과

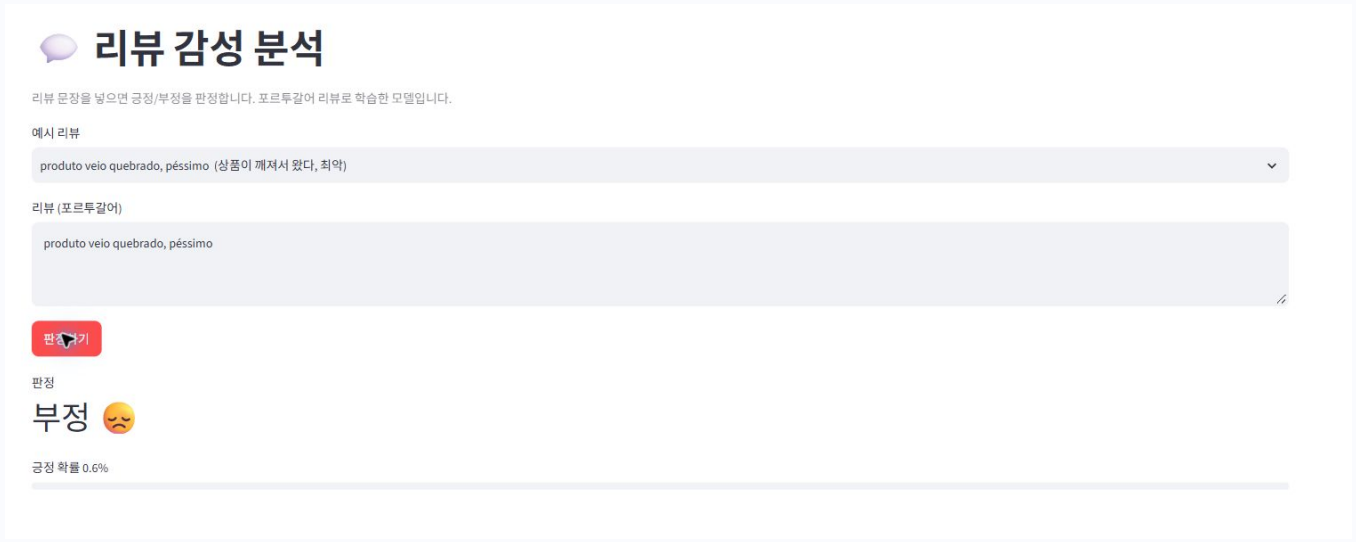
문서화 중 실제 화면 확인



실제 화면 확인. 분석 파일 재생성 후 KPI와 그래프를 확인했다.

문서화 중 확인한 기능: 홈, 재생성 데이터의 분석 KPI와 탭, 감성 예문 판정, 추천 5개, 판매자 유형 예시 판정, 이탈 위험 지표. 모든 입력 조합, 경계값, 동시 접속, 외부 배포를 시험한 것은 아니다.

문서화 중 실제 화면 확인



실제 화면 확인. 부정 예문의 긍정 확률은 약0.6%다.

⑥ 결과 해석

홈, 분석 KPI와 탭, 감성 예문, 추천 5개, 유형 예시, 이탈 지표는 확인했다. RAG 화면의 API 키 대기 상태는 새 챗봇 응답 성공과 다르다. 모든 입력 조합과 외부 배포는 미검증이다.

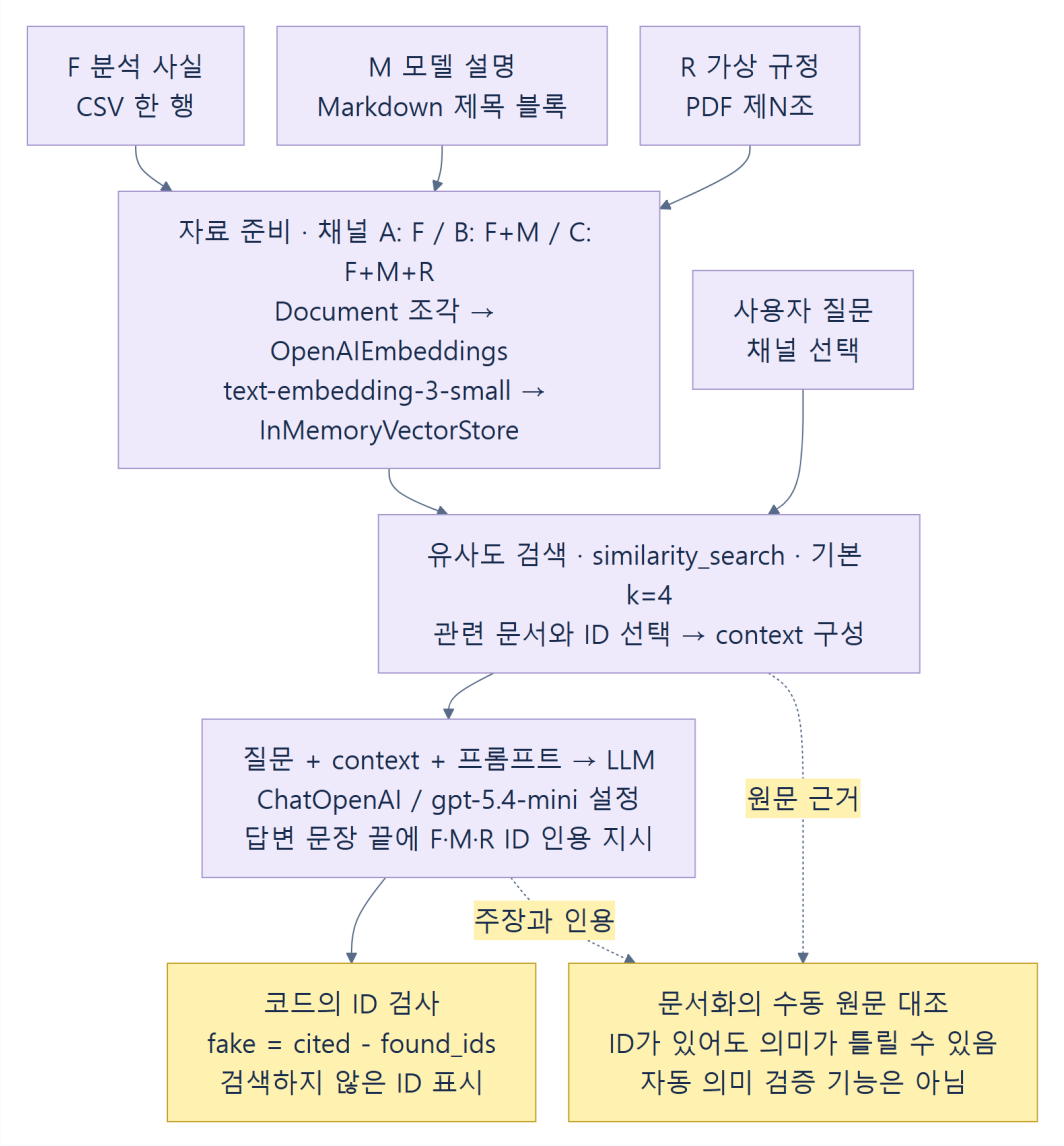
⑦ 이번 단계에서 배운 점

화면의 기능 확인과 데이터 계산의 정확성은 각각 검증해야 한다. 메뉴가 있다는 것만으로 해당 기능의 실행 완료를 주장할 수 없다.

구조도 03 검색과 인용 검증

자료 검색부터 인용 검증까지의 RAG 구조

F는 분석 사실, M은 모델의 입력·작동·한계 설명, R은 실습용 가상 규정이다. 서로 다른 종류의 문서를 채널별로 선택해 검색한다.



같은 Mermaid 원본에서 로컬 렌더링한 고해상도 구조도. 화살표와 관계는 실제 파일·코드를 대조했다.

실선은 구현된 문서 준비·검색·답변·ID 검사의 흐름이다. 점선은 문서화 중 사람이 수행한 원문 대조이며 자동 의미 검증 기능이 아니다. cited - found_ids는 검색 결과에 없는 ID만 찾는다. 검색된 ID를 잘못 인용하거나 인용을 빠뜨린 경우까지 보장하지 않는다.

근거: utils/rag.py의 load_fact_docs, load_guide_docs, load_policy_docs, build_store, ask 및 11_olist_rag_chatbot.ipynb 셀 5·7·9·12·15. API 모델명은 파일 설정을 표시한 것이며 새 API 호출·응답 생성은 하지 않았다.

STEP 18 검색 기반 답변

RAG가 근거를 찾아 답하도록 구성한 과정

① 실습 목적

질문을 받으면 관련 자료를 먼저 찾고 그 근거로 답하도록 구성한다.

② 왜 이 작업이 필요한가

언어 모델이 기억에만 의존하면 프로젝트에 없는 숫자나 규정을 만들 수 있다. 답변할 자료 범위를 정하고 원문을 함께 보여 줄 필요가 있다.

③ 사용한 기술과 코드

검색 증강 생성(Retrieval-Augmented Generation, RAG)은 자료 검색을 답변 생성에 연결하는 방식이다. 벡터 검색(Vector Search)은 임베딩으로 바꾼 질문과 자료의 숫자 표현이 얼마나 가까운지 비교한다. Olist에서는 사실표·모델 설명·가상 규정에서 관련 조각을 찾는다.

```
# 해설: 관련 자료를 먼저 찾고 그 문맥으로 답한다.
store = build_store(channel, api_key)
found = store.similarity_search(question, k=k)
context = "\n".join(
    f"({d.metadata['id']}) {d.page_content}" for d in found)
answer = (PROMPT | llm | StrOutputParser()).invoke(
    {"context": context, "question": question})
```

핵심 코드 발췌 · 앞뒤 코드 생략
utils/rag.py · ask 함수 일부 · 해설 주석 추가, 계산 로직 유지

```
# 해설: 관련 자료를 먼저 찾고 그 문맥으로 답한다.
found_ids = {d.metadata["id"] for d in found}
cited = set(re.findall(r"\[([FMR]\d{2})\]", answer))
fake = sorted(cited - found_ids)
```

핵심 코드 발췌 · 앞뒤 코드 생략
utils/rag.py · 근거 ID 비교 로직 발췌 · 해설 주석 추가, 계산 로직 유지

④ 실습 과정

1. utils/rag.py의 load_fact_docs·load_guide_docs·load_policy_docs를 확인했다.
2. build_store에서 임베딩과 저장소를 만들고 ask에서 기본 4개를 찾는 코드를 읽었다.
3. 답변 프롬프트와 근거 ID 비교 로직을 대조했다. 새 API 호출은 하지 않았다.

⑤ 실행 결과

채널	검색 자료	확인 범위
A	사실표 35행	분석 수치
B	사실표와 모델 설명 8개	수치와 모델의 입력·한계
C	위 자료와 가상 규정 PDF	추가 운영 규정

CSV 한 행, 모델 설명서의 제목 한 개, 규정집의 조항 한 개를 문서 단위로 사용한다. 임베딩 모델은 text-embedding-3-small, 코드의 답변 모델 설정은 gpt-5.4-mini이며 이는 파일 설정을 기록한 것이다. 이번에 제공 여부를 조회하거나 새 API 응답을 생성하지 않았다.

⑥ 결과 해석

A는 사실표, B는 설명서 추가, C는 가상 규정 추가다. 코드의 모델명은 설정 기록이지 최신 제공 상태를 확인한 결과가 아니다.

⑦ 이번 단계에서 배운 점

RAG는 자료를 찾아 쓰는 과정이지 답을 보증하는 장치가 아니다. 검색된 자료가 질문에 맞는지와 답이 원문에 충실한지를 따로 확인한다.

STEP 19 팩트체크

사실 3개와 챗봇 질문을 원문과 대조한 결과

① 실습 목적

사실 3개와 챗봇 답변을 원본 데이터·문서와 비교한다.

② 왜 이 작업이 필요한가

답 끝에 번호가 붙어 있으면 신뢰하기 쉽지만, 번호가 잘못됐거나 답 자체가 자료 범위를 벗어날 수 있다.

③ 사용한 기술과 코드

사실표(Fact Sheet)는 확인한 사실을 ID·수치·기준과 함께 적은 표다. 팩트체크(Fact-checking)는 주장과 근거를 대조하는 작업이다. 여기서는 F01·F07·F14를 CSV와 비교하고 규정 질문은 R06 원문을 읽었다.

④ 실습 과정

1. 사실표의 ID와 분석 기준을 확인했다.
2. 답 있는 질문 2개와 답 없는 질문 1개를 기존 제출 화면에서 읽었다.
3. 숫자·출처 ID·자료에 없는 내용의 처리 여부를 각각 판정했다. 새 응답은 만들지 않았다.

⑤ 실행 결과

사실 ID	확인한 사실	대상과 기준
F01	분석 주문 96,203건	배송 완료·수령일 존재·분석 기간
F07	재구매 고객 비율 3.0%	customer_unique_id별 2회 이상 주문
F14	지연 배송 평균 리뷰 2.57점	분석 대상 중 지연 주문, 주문별 첫 리뷰

질문	확인한 답 또는 기대 범위	근거와 판정
늦게 도착한 주문의 평균 리뷰는?	기존 제출 화면에 2.57점 답변	F14: 숫자 재계산과 일치
배송이 3일 이상 늦으면 보상은?	고객 요청 시 배송비 환불, 책임 주체 부담	가상 규정 R06 원문과 대조. 실제 기업 정책 아님
2019년 Olist 매출은?	기존 비교 화면에 확인 불가 답변	해당 연도 근거 없음. 일부 화면의 불필요한 F02·F19 인용은 부적절

세 사실은 기존 사실표 ID를 유지했다. F01·F07·F14의 핵심 값은 로컬 CSV로 다시 계산했다. R06은 분석 사실이 아니라 수업용 가상 규정 문서의 조항 ID다.

이번 작업에서는 외부 API를 다시 호출하지 않았다. 평가 Word의 삽입 화면과 원문을 대조한 결과이며, 현재 코드의 새 실행 결과로 표시하지 않았다.

⑥ 결과 해석

F14의 2.57점은 재계산과 일치한다. R06은 실제 Olist 정책이 아니라 가상 규정이다. 2019년 매출은 근거가 없으며 확인 불가 답변에 무관한 ID가 붙은 버전은 인용이 부적절하다.

⑦ 이번 단계에서 배운 점

답변의 내용과 인용의 정확성은 별개다. 자료에 없는 숫자를 만들지 않고 확인 불가라고 말하는 것도 올바른 결과다.

STEP 20 RAG 증빙

실제 제출 화면에서 발견한 근거 오류

① 실습 목적

실제 제출 화면에서 답변 내용과 근거 ID가 어긋난 부분을 찾는다.

② 왜 이 작업이 필요한가

기자에게는 문장이 그럴듯한지보다 원문이 실제로 그 주장을 뒷받침하는지가 중요하다.

③ 사용한 기술과 코드

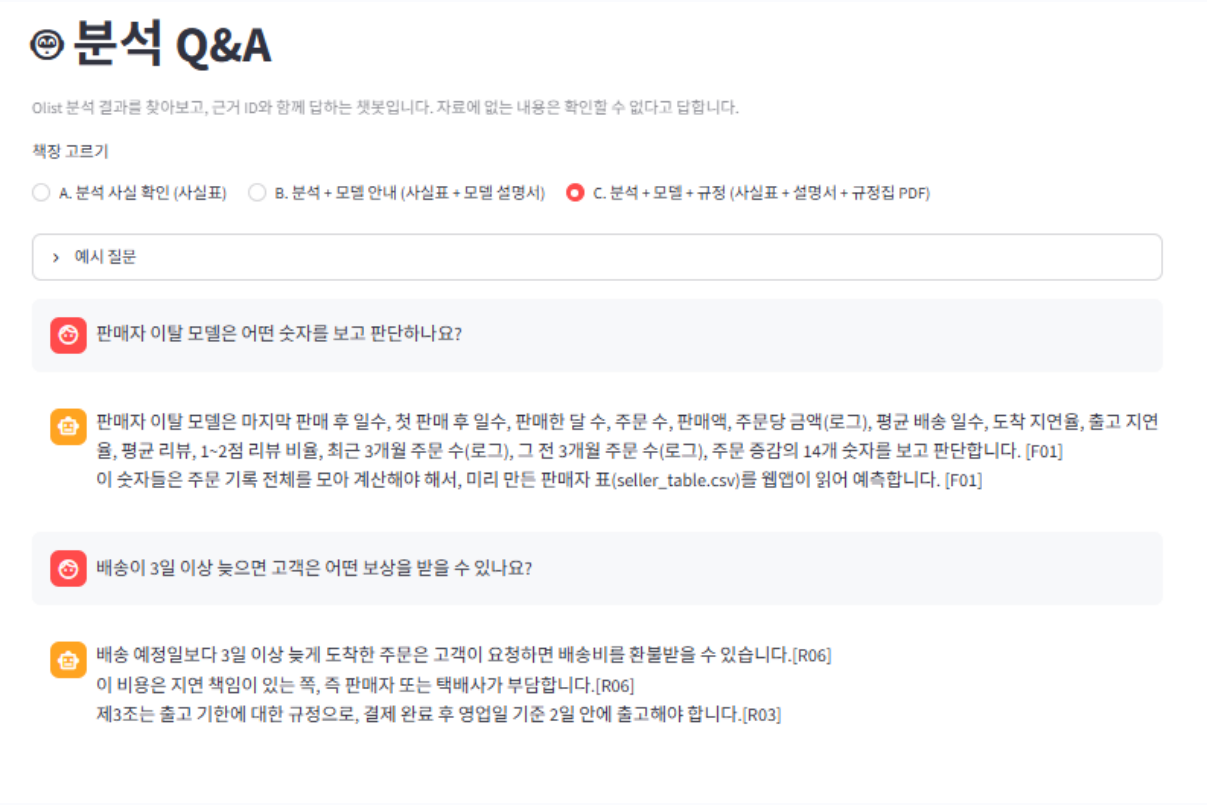
집합 차집합(Set Difference)은 한 목록에는 있고 다른 목록에는 없는 값을 찾는 방법이다. cited - found_ids는 검색하지 않은 ID를 찾지만, 이미 검색된 엉뚱한 ID의 의미까지 판정하지는 못한다.

④ 실습 과정

1. 평가 Word의 채널 C 화면을 읽었다.
2. 모델 입력 설명에 붙은 F01과 실제 모델 설명 M09를 비교했다.
3. 다른 비교표의 F11 인용과 가상 환불 조항 R06도 대조했다.

⑤ 실행 결과

기존 평가 문서 화면



평가 Word에 삽입된 채널 C 화면. 모델 설명에 F01을 붙인 부분과 보상 규정 R06을 함께 확인할 수 있다.

⑥ 결과 해석

F01은 분석 주문 수이므로 모델 입력 14개 설명의 출처로 부적절하다. 보상 규정은 R06을 확인해야 한다. fake 목록이 비어 있어도 의미상 올바른 인용이라고 단정할 수 없다.

⑦ 이번 단계에서 배운 점

검증 코드는 확인할 범위를 가진다. 코드로 ID 존재를 확인한 뒤 사람이 문장의 의미와 원문을 대조하는 과정이 필요하다.

STEP 21 채널 비교

세 채널을 비교한 기록과 답변 범위

① 실습 목적

자료 묶음 A·B·C에 따라 답할 수 있는 범위가 달라지는지 살펴본다.

② 왜 이 작업이 필요한가

규정집을 넣지 않은 챗봇에게 보상 규정을 물으면 근거가 없다. 질문에 필요한 문서가 있는지부터 확인해야 한다.

③ 사용한 기술과 코드

문서 조각(Document Chunk)은 검색할 수 있도록 나눈 자료 단위다. 이 프로젝트는 CSV 한 행·모델 설명 제목 한 개·PDF 조항 한 개를 사용해 의미가 끊어지지 않도록 구성한다.

④ 실습 과정

1. 평가 Word의 기존 채널 비교표를 확대해 읽었다.
2. 수치·모델·규정 질문마다 필요한 문서 층을 연결했다.
3. 저장 노트북의 조각 수 설명과 이번 확인 범위를 구분했다.

⑤ 실행 결과

기존 평가 문서 화면

	질문	B. +설명서
0	늦게 도착한 주문의 리뷰 점수는?	늦게 도착한 주문의 평균 리뷰 점수는 2.57점입니다. [F14]
1	재구매 고객은 몇 %야?	재구매한 고객은 3.0%입니다. [F07]
2	판매자 유형은 몇 가지이고 어떻게 나눴어?	판매자 유형은 4가지이며, 핵심 우량·신규 성장·휴면·저평점 위험으로 나눕니다.[F27] \n이 구분은 판매자 숫자 5개(주문 건수, 판매액, 마지막 판매 후 지난 날 수, 첫 판매 후 지난 날 수, 평균 리뷰 점수)를 사용해 K-Means 군집으로 비슷한 판매자끼리 4묶음으로 나눈 것입니다.[M08]
3	이탈 모델은 어떤 숫자를 보고 판단해?	이탈 모델은 마지막 판매 후 일수, 첫 판매 후 일수, 판매한 달 수, 주문 수·판매액·주문당 금액(로그), 평균 배송 일수, 도착 지연율, 출고 지연율, 평균 리뷰, 1~2점 리뷰 비율, 최근 3개월·그 전 3개월 주문 수(로그), 주문 증감의 14개 숫자를 보고 판단합니다. [M09] \n이 숫자들은 주문 기록 전체를 모아 계산해야 해서 판매자 표(seller_table.csv)로 미리 만들어 두고 웹앱은 그 표를 읽어 예측합니다. [M09]
4	배송이 3일 늦으면 고객은 무엇을 받을 수 있어?	제공된 자료로는 확인할 수 없습니다.
5	저평점 위험 판매자는 어떤 조치를 받아?	저평점 위험 판매자는 평균 배송일이 약 22일이고 도착 지연율 약 31%, 출고 지연율 약 33%, 1~2점 리뷰 비율이 80%를 넘는 판매자입니다 [F29] \n따라서 이 유형은 리뷰·배송 지연 문제를 중심으로 관리 대상이 되는 것으로 볼 수 있지만, 구체적인 조치 내용은 제공된 자료로는 확인할 수 없습니다. [F29]
6	2019년 매출은 얼마야?	제공된 자료로는 확인할 수 없습니다.

기존 평가 Word 비교표의 질문 열과 채널 B 열을 붙여 확대했다. 원문 내용은 수정하지 않았다.

기존 평가 문서 화면

	질문	C. +규정집	가짜 ID
0	늦게 도착한 주문의 리뷰 점수는?	늦게 도착한 주문의 평균 리뷰 점수는 2.57점입니다. [F14]	[]
1	재구매 고객은 몇 %야?	재구매 고객은 3.0%입니다. [F07]	[]
2	판매자 유형은 몇 가지이고 어떻게 나눴어?	판매자 유형은 4가지이며, 핵심 우량 876명, 신규 성장 947명, 휴면 461명, 저평점 위험 218명으로 나눴습니다. [F27] \n이 구분은 판매자 숫자 5개(주문 건수, 판매액, 마지막 판매 후 지난 날 수, 첫 판매 후 지난 날 수, 평균 리뷰 점수)를 바탕으로 K-Means 군집으로 4묶음으로 나눈 것입니다. [F27][M08] \n주문 건수와 판매액은 로그로 바꾸고, 리뷰가 없는 판매자는 학습 때 중앙값(review_fill)으로 채웠습니다. [M08]	[]
3	이탈 모델은 어떤 숫자를 보고 판단해?	이탈 모델은 마지막 판매 후 일수, 첫 판매 후 일수, 판매한 달 수, 주문 수·판매액·주문당 금액(로그), 평균 배송 일수, 도착 지연율, 출고 지연율, 평균 리뷰, 1~2점 리뷰 비율, 최근 3개월·그 전 3개월 주문 수(로그), 주문 증감의 14개 숫자를 보고 판단합니다. [M09] \n이 숫자들은 주문 기록 전체를 모아 계산해야 해서, 미리 만든 판매자 표(seller_table.csv)를 읽어 예측합니다. [M09]	[]
4	배송이 3일 늦으면 고객은 무엇을 받을 수 있어?	배송 예정일보다 3일 이상 늦게 도착한 주문의 고객이 요청하면 배송비를 환불받을 수 있습니다. [F11] 비용은 지연 책임이 있는 쪽(판매자 또는 택배사)이 부담합니다. [F11]	[]
5	저평점 위험 판매자는 어떤 조치를 받아?	저평점 위험 판매자는 제13조에 따라 출고 기한 준수 계획서를 14일 안에 제출해야 합니다. [R13] \n다음 분기에도 저평점 위험 등급이면 신규 상품 등록이 제한됩니다. [R13]	[]
6	2019년 매출은 얼마야?	제공된 자료로는 확인할 수 없습니다. [F19]	[]

같은 표의 질문 열과 채널 C 열. 배송 보상의 F11 인용과 확인 불가 답변의 F19 인용을 원문과 대조한다.

저장 노트북에는 A 35개, B 43개, C 60개 조각이라고 설명돼 있다. 35행 사실표와 설명서 8개는 이번에 확인했다. C의 최종 로더 결과 조각 수는 새 실행으로 확인하지 않았으므로 60을 실측 결과로 제시하지 않았다.

⑥ 결과 해석

35행 사실표와 설명서 8개는 확인했다. 노트북 설명의 C 60개 조각은 로더를 새로 실행한 실측 결과가 아니다. 관련 숫자를 찾았어도 질문에 충분히 답하지 못한 버전이 있다.

⑦ 이번 단계에서 배운 점

자료를 많이 넣는 것만으로 검색 품질이 보장되지는 않는다. 질문에 맞는 자료와 충분한 답을 골랐는지를 함께 살펴야 한다.

STEP 22 문제 해결

오류와 수정 과정을 실제 확인 범위로 기록

① 실습 목적

실제로 관찰한 오류를 원인 판단·대응·확인 결과로 나누어 기록한다.

② 왜 이 작업이 필요한가

오류를 겪었다는 말만으로는 다음에 재현하거나 해결하기 어렵다. 확인한 사실과 아직 추정인 원인을 나눠야 한다.

③ 사용한 기술과 코드

역직렬화(Deserialization)는 저장된 바이트를 표 같은 프로그램의 구조로 되돌리는 일이다. 원본 Parquet의 오류는 이 단계에서 발생했다. 예외 처리(Exception Handling)는 오류나 빈 입력에서 중단·안내하는 흐름을 정하는 코드다.

④ 실습 과정

1. 원본 분석 메뉴의 오류 문구와 화면을 보존했다.
2. 준비 노트북 규칙을 사용해 별도 분석 파일을 재생성했다.
3. 재독출·지표·정상 화면을 대조했다. RAG ID 문제는 문서 대조로 확인했고 코드 수정 완료로 쓰지 않았다.

⑤ 실행 결과

관찰	원인 또는 판단	대응과 결과
orders.parquet 읽기 실패	Thrift 메타데이터 역직렬화 오류. 발생 원인 미확정	원본 보존, 별도 복사본에서 준비 노트북 규칙으로 재생성. 다시 읽기 및 화면 KPI 확인
종료 날짜에 따른 11건 차이	8월29일 자정 이하와 8월30일 미만의 차이	수업 원문 조건을 적용해 96,203건 재현
F01 F11 근거 오류	답변 내용과 인용 문서의 주제가 어긋남	기존 제출 화면을 모델 설명 M09, 규정 R06 원문과 대조. 코드 수정 완료라고 쓰지 않음
과거 환경과 현재 환경 차이	저장 출력은 Python3.13, 로컬 확인은 3.12	현재 실행 버전을 별도 기록. 재학습 성능 동일성은 미확인

분석 화면의 빈 데이터 방어 코드는 이미 views/analysis.py에 존재한다. 이전 기술 요약에서 언급한 빈 필터 오류 가능성을 현재 확인된 결함으로 확정하지 않았다. 해결했다는 기록은 실제 수정과 재확인이 있을 때만 남긴다.

문서화 중 실제 화면 확인



문서화 중 실제로 캡처한 원본 분석 메뉴 오류 화면. 과거 사용자 실습 오류라고 추정하지 않았다.

⑥ 결과 해석

파일 재생성 후 분석 지표는 일치했다. 날짜 경계 오류는 수업 원문 조건으로 바로잡았다. 빈 데이터 방어 코드는 원래 존재했으므로 새로 고쳤다고 기록하지 않았다.

⑦ 이번 단계에서 배운 점

> 해결했다는 말 뒤에는 확인 결과가 있어야 한다. 문서화 과정에서 발견한 일을 과거 본인 실습 경험으로 바꾸지 않는 것도 정확한 기록이다.

STEP 23 학습 성찰

단계별로 정리한 배운 점과 활용 가능성

① 실습 목적

각 실습에서 이해한 개념과 앞으로 적용할 곳을 정리한다.

② 왜 이 작업이 필요한가

도구 이름이나 성능 수치만 기억하면 다른 문제에 적용하기 어렵다. 어떤 판단을 했고 무엇을 조심해야 하는지 자신의 말로 설명할 수 있어야 한다.

③ 사용한 기술과 코드

이 단계는 새 모델이나 코드를 실행한 단계가 아니다. 기존 코드·계산·화면·오류 기록을 연결해 데이터 저널리즘 관점에서 의미를 정리했다.

④ 실습 과정

1. 병합과 집계에서 배운 점을 정리했다.
2. 모델 평가와 서비스 연결의 판단을 비교했다.
3. RAG 근거 검증과 업무 활용 가능성을 정리하고 아직 구현하지 않은 개선안은 제안으로 표시했다.

⑤ 실행 결과

실습	정리한 학습 내용	활용 예
병합과 집계	행 수와 고유 주문 수를 구분해야 중복 계산을 막는다	음식 사업의 주문·메뉴·영수증 집계
EDA	분석 기간과 분모를 먼저 정하고 상관과 인과를 구분한다	데이터 기사에서 통계 기준 공개
모델 평가	정확도 외에 불균형과 재현율·정밀도, 시험 분리를 본다	위험 경보의 누락과 헛경보 비용 비교
저장과 서비스	전처리와 모델을 묶어야 입력 처리가 재현된다	분석 결과를 팀이 쓰는 간단한 화면으로 제공
RAG	검색된 ID와 답변 의미를 별도로 검증한다	자료 조사와 기사 초안의 근거 점검

강사 제공 코드와 교육 자료를 바탕으로 실습 흐름과 저장 출력을 분석했다. 이번 문서화에서는 AI 도구의 도움을 받아 ZIP·로컬 대조, 핵심 지표 재계산, 일부 모델 추론, 화면 캡처, 분석 데이터 복구와 문서 편집을 수행했다. 문서화 중 추가 확인을 과거에 혼자 수행한 일로 서술하지 않았다.

실제 사용자 로그로 추천의 클릭·구매 성과를 평가하고, 이탈 경보는 시간 순서 검증과 확률 보정을 검토할 수 있다. RAG는 미인용 답변·부적절한 인용의 의미 검증을 보강하고 외부 입력의 영향을 점검할 필요가 있다. 이는 후속 제안이며 구현 완료가 아니다.

⑥ 결과 해석

실제 사용자 추천 클릭·구매 성과, 캠페인의 매출 효과, 새 RAG API 응답은 이번 확인 결과에 없다. 강사 제공 코드와 AI 도구의 문서화 지원 범위를 명시했다.

⑦ 이번 단계에서 배운 점

숫자에는 기준이, 모델에는 평가 조건이, AI 답변에는 적절한 원문 근거가 필요하다. 제출자는 이 내용을 자신이 설명할 수 있는 표현으로 검토하고 수정할 수 있다.

제출 체크

완료 범위와 제출 전 확인 사항

항목	상태	근거
ZIP과 로컬 핵심 파일	확인	84개 CRC 대조
데이터 크기·결측치·핵심 지표	재계산	runtime_verification.json / verified_metrics.json
추천·감성·군집·이탈 학습 평가	저장 출력 확인	ZIP 노트북의 셀과 출력
모델 새 학습	미실행	새 재학습 성능이라고 주장하지 않음
저장 모델 일부 추론	재계산·화면 확인	감성 예문·판매자 이탈·추천·유형 화면
Streamlit 분석	복구 후 확인	별도 복사본 Parquet 재생성 및 KPI 대조
Gradio 새 화면	미실행	노트북 과거 출력만 확인
RAG 새 API 호출	미실행	기존 제출 화면과 원문 대조
Drive 15장·참고17장	확인	구조 증거와 타인 참고자료로 분리

제출자가 마지막으로 확인할 것

이름과 제출일, 실제 본인 수행 범위, 평가 양식의 (1) 사본과 확보본 차이, Olist가 문제1 대체 자료로 허용되는지 확인한다. 과거 실행 화면과 이번 확인용 화면을 혼동하지 않는다. API 키나 개인 인증정보는 결과물에 넣지 않았다.

현재 PDF는 저장된 코드·출력·기존 제출 화면과 이번 추가 확인을 연결한 실습 포트폴리오다. 전체 앱의 모든 상황에서 정상 작동하거나 실제 사업 성과가 개선됐다는 보증 문서는 아니다.

부록 · 별도 제출 확인 사항

강사 평가 양식과 연결한 제출 항목

확보한 파일은 5차_AI-Powered_Python_자동팩트체킹_김형석.docx다. 요청에서 언급한 (1) 사본과의 동일성은 확인하지 못했으므로, 아래는 확보한 양식 기준의 대응표다. 양식의 평가는 총 100점이며 두 문항으로 구성된다.

문항	양식 요구사항	포트폴리오에 반영한 근거
문제1 50점	데이터 선택, 행·열, 결측치, 계산, 사실 3개, 코드·결과 캡처	Olist 원본 규모·결측치 재계산, 사실표와 분석 코드·그래프
문제2 50점	답 있음 2개와 답 없음 1개, 답·ID·판정, 질문 답변 캡처	F14 리뷰 점수, R06 가상 보상 규정, 2019년 매출 확인 불가; 기존 제출 화면과 대조
한 줄 정리	AI 답을 그대로 쓰면 안 되는 이유	내용과 근거 ID가 어긋난 F01·F11 사례를 원문과 비교

데이터 선택 범위의 차이

양식 문제1의 선택지는 타이타닉, 대기오염, auto_mpg다. Olist는 이 목록에 없으므로 본 포트폴리오가 문제1을 공식적으로 대체할 수 있는지는 강사 확인이 필요하다. 원래 양식에 작성된 타이타닉 답안은 수정하거나 Olist로 바꿨다고 주장하지 않았다.

양식에는 타이타닉 891행×15열, age 결측 177개 등 답안과 Olist RAG 기록이 함께 있다. 삽입 이미지 중에는 다른 스키마의 타이타닉 Cabin 열 및 Iris 화면도 포함돼 있어, 이를 Olist 분석의 실행 증거로 재사용하지 않았다.

제출용 Olist 포트폴리오는 상세 실습 증빙으로 완성했다. 공식 양식의 허용 데이터와 문항별 제출 범위는 분리해 확인해야 한다.

참고 자료

자료 출처와 증빙 위치

출처	사용 위치와 성격
Olist.zip	STEP1~4 노트북, 앱 코드, CSV, 모델, RAG 자료. ZIP 내 가상환경과 Git은 내용 분석 대상에서 제외
5차 평가 Word	평가 요구사항, 기존 Olist RAG 대화와 채널 비교 화면. 다른 데이터 예시는 Olist 결과로 전용하지 않음
Drive 15장	Data_Analysis 2장, Olist 루트, 상위 model·rag·data, STEP1·STEP2·STEP3·STEP4, dashboard와 하위4개 폴더
참고17장 094949~095128	타인 GitHub README·포트폴리오의 흐름, 정보 위계, 차트·설명 조합을 참고. 타인 서비스 링크와 숫자는 결과에 미사용
이번 확인 로그	verified_metrics.json, runtime_verification.json, zip_local_comparison.json, notebook_evidence.json
캡처와 GIF	assets/streamlit-*.png, assets/streamlit-demo.gif. 분석 화면은 복구한 복사본에서 확인

외부 데이터와 교육 자료의 표시

원본 데이터는 수업에서 사용한 Olist 브라질 이커머스 공개 데이터다. ZIP의 다운로드 노트북은 KaggleHub를 통해 olistbr/brazilian-ecommerce를 참조한다. 본 문서는 외부 웹사이트의 최신 내용이나 라이선스를 별도 조회하지 않았으며, 공개 저장소 배포 시 원 데이터의 배포 조건을 확인해야 한다.

규정집 olist_seller_policy.pdf는 첫 페이지에 실습용 가상 규정임이 명시돼 있다. 문서에 나오는 환불·출고·등급 조항을 실제 Olist의 공식 정책으로 안내하지 않는다.

재현 위치: C:/WORKSPACE/Olist · Drive 위치: My Drive/Colab Notebooks/Data_Analysis/Olist. 이 문서 제작 과정에서 원본 프로젝트와 동기화 sources는 수정하지 않았다.

최종 성과 요약

전체 실습 흐름과 기능별 검증 상태

데이터 분석 → 상품 추천 → 리뷰 감성 분석 → 판매자 군집·이탈 예측 → Streamlit → RAG 팩트체크

서로 다른 표를 연결해 질문을 만들고, 저장 모델의 평가를 해석한 뒤 화면과 답변의 근거까지 확인한 실습이다. 아래 상태는 기존 증거의 요약이며 새로운 실행 성과를 추가한 것이 아니다.

실습 기능	검증 상태	확인한 결과와 범위
데이터 분석	CSV 재계산·화면 확인	96,203건, 결제 합계 R$15,374,496.52. 집계 단위와 날짜 기준을 맞춤
상품 추천	저장 평가·추론·화면 확인	KNN 예시 추천 5개 확인. 72%·45%·30%는 특징 차이이며 추천 정확도가 아님
리뷰 감성 분석	저장 평가·ML 예시 추론 확인	기본 ML 0.917, 개선 0.926, DL 0.9235는 저장 평가. 이번 재학습 아님
판매자 군집·이탈	저장 평가·일부 추론·화면 확인	4개 군집. 이탈 시험 정밀도 0.435·재현율 0.750·F1 0.551. 최신 대상 1,817명 중 위험 637명
Streamlit	로컬 실제 화면 확인	홈·감성·추천·이탈 확인. 분석·유형은 확인용 복사본에서 확인. 분석 파일 재생성
RAG 팩트체크	기존 제출 화면·원문 대조	F14의 2.57점 재계산 일치. F01·F11 등의 부적절한 인용 확인. 새 API 호출 미실행

새로 수행하지 않은 일: 모델 재학습, Gradio 새 실행, RAG 새 API 호출, 모든 입력 조합 시험, GitHub Push와 외부 배포. 가상 규정은 실제 Olist 정책이 아니다.

배운 점: 숫자의 집계 기준, 모델의 평가 조건, 화면의 동작과 답변의 인용 근거를 각각 확인해야 결과를 정확히 설명할 수 있다.